ಅಸ್ತವ್ಯಸ್ತ ಡೇಟಾದಿಂದ ಕೆಲಸ ಮಾಡುವ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ವರೆಗೆ

ಪ್ರಯೋಗ ಕಾರ್ಯ (Lab) ೧ — ಸಂಪೂರ್ಣ ಕನ್ನಡ ಮಾರ್ಗದರ್ಶಿ

ಡೇಟಾ ಸ್ವಚ್ಛಗೊಳಿಸುವಿಕೆ → ಅಸಹಜ ಮೌಲ್ಯ ಪತ್ತೆ → ಸ್ಕೇಲಿಂಗ್ → ಸಣ್ಣ ನೆಟ್‌ವರ್ಕ್ → MNIST ಅಂಕೆಗಳು → ಹೋಲಿಕೆ

“Lab 1 Visual Guide” (VGTU · Neural Networks) ಆಧರಿಸಿದ ಕನ್ನಡ ಅಧ್ಯಯನ ಪುಸ್ತಿಕೆ
ನೈಜ ಉದಾಹರಣೆಗಳು, ಹಂತ-ಹಂತದ ಲೆಕ್ಕಗಳು, ಕೋಡ್ ಮತ್ತು ವೈವಾ ಪ್ರಶ್ನೋತ್ತರ ಸಹಿತ

ಪರಿವಿಡಿ

೦. ದೊಡ್ಡ ಚಿತ್ರ — ಈ ಲ್ಯಾಬ್ ಒಂದು ಕಥೆ

೧. ನಿಜ ಜೀವನದಲ್ಲಿ ಇದು ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ? (ಆಸ್ಪತ್ರೆ, ಕ್ರೀಡೆ, ಅಂಚೆ ಕಚೇರಿ)

೨. ಕಾರ್ಯ ೧ — ಕಾಣೆಯಾದ ಮೌಲ್ಯ ತುಂಬುವುದು (Imputation)

೩. ಕಾರ್ಯ ೨–೪ — ಅಸಹಜ ಮೌಲ್ಯ ಪತ್ತೆ (Z-score, Chauvenet, IQR)

೪. ಸಾಮಾನ್ಯೀಕರಣ (Normalization) — ಎಲ್ಲರಿಗೂ ಒಂದೇ ಅಳತೆಪಟ್ಟಿ

೫. ಕಾರ್ಯ ೫ — ಕೈಯಿಂದ ಬರೆದ ಸಣ್ಣ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ (ಒಂದು ಹೆಜ್ಜೆಯ ಪೂರ್ಣ ಲೆಕ್ಕ)

೬. ಕಲಿಕೆಯ ವೇಗ (Learning rate) — ಬೆಟ್ಟ ಇಳಿಯುವ ಉದಾಹರಣೆ

೭. ಕಾರ್ಯ ೬ — MNIST ಅಂಕೆಗಳನ್ನು ಓದುವ ನೆಟ್‌ವರ್ಕ್

೮. ಕಾರ್ಯ ೬.೨ — ಆಕ್ಟಿವೇಶನ್ ಫಂಕ್ಷನ್‌ಗಳ ಹೋಲಿಕೆ

೯. ಕಾರ್ಯ ೬.೩ — ಆಪ್ಟಿಮೈಜರ್ ಮತ್ತು ಲರ್ನಿಂಗ್ ರೇಟ್

೧೦. ನನ್ನ ಪ್ರಯೋಗ ಫಲಿತಾಂಶಗಳು ಒಂದೇ ನೋಟದಲ್ಲಿ

೧೧. ಲ್ಯಾಬ್‌ಗೆ ಬೇಕಾದ ಕೋಡ್ (Python)

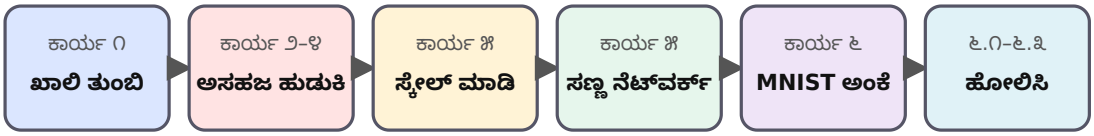
೧೨. ಏನೆಲ್ಲಾ ಕಲಿಯಬೇಕು — ಪರಿಶೀಲನಾ ಪಟ್ಟಿ

೧೩. ವೈವಾ (Defense) ಪ್ರಶ್ನೋತ್ತರಗಳು

> **ಈ ಪುಸ್ತಿಕೆಯನ್ನು ಹೇಗೆ ಓದಬೇಕು?**
>
> ಮೂಲ “Visual Guide” ಒಂದು HTML ಪುಟ — ಅದರಲ್ಲಿ ಸ್ಲೈಡರ್‌ಗಳು, ಲೈವ್ ಡೆಮೊಗಳು ಇವೆ. ಇಲ್ಲಿ PDF ಆಗಿರುವುದರಿಂದ ಆ ಡೆಮೊಗಳ ಬದಲು **ಚಿತ್ರಗಳು ಮತ್ತು ಕೈಯಿಂದ ಮಾಡಿದ ಲೆಕ್ಕಗಳು** ಇವೆ. ಪ್ರತಿ ಭಾಗದಲ್ಲಿ ಮೊದಲು “ಯಾಕೆ?” (ನಿಜ ಜೀವನದ ಕಥೆ), ನಂತರ “ಹೇಗೆ?” (ಲೆಕ್ಕ), ಕೊನೆಯಲ್ಲಿ “ಲ್ಯಾಬ್‌ನಲ್ಲಿ ಏನಾಯಿತು?” ಎಂದು ಇದೆ. ಲ್ಯಾಬ್‌ನ ನಿಜವಾದ ಫಲಿತಾಂಶಗಳನ್ನು (ಉದಾ: 98.16%) ಮೂಲ ಗೈಡ್‌ನಿಂದಲೇ ತೆಗೆದುಕೊಂಡಿದ್ದೇನೆ; ಉಳಿದ ಚಿಕ್ಕ ಉದಾಹರಣೆಗಳ ಸಂಖ್ಯೆಗಳನ್ನು Python ನಲ್ಲಿ ಲೆಕ್ಕ ಹಾಕಿ ಪರಿಶೀಲಿಸಿದ್ದೇನೆ.

೦. ದೊಡ್ಡ ಚಿತ್ರ — ಈ ಲ್ಯಾಬ್ ಒಂದು ಕಥೆ

ಈ ಲ್ಯಾಬ್‌ನಲ್ಲಿ ಆರು ಕಾರ್ಯಗಳಿವೆ. ಆದರೆ ಅವು ಆರು ಬೇರೆ ಬೇರೆ ವಿಷಯಗಳಲ್ಲ — ಒಂದೇ ಕಥೆಯ ಆರು ಅಧ್ಯಾಯಗಳು. ನಿಜವಾದ ಮೆಷಿನ್ ಲರ್ನಿಂಗ್ ಯೋಜನೆಗಳಲ್ಲೂ ಇದೇ ಕ್ರಮ: **ಮಾಡೆಲ್ ಮಾಡುವುದು ಕೆಲಸದ ಕೊನೆಯ ಮೂರನೇ ಒಂದು ಭಾಗ ಮಾತ್ರ**; ಉಳಿದ ಸಮಯ ಡೇಟಾ ಸರಿಪಡಿಸುವುದಕ್ಕೇ ಹೋಗುತ್ತದೆ.



ಚಿತ್ರ ೦.೧: ಲ್ಯಾಬ್‌ನ ಆರು ಕಾರ್ಯಗಳು — ಒಂದರ ಹಿಂದೆ ಒಂದು

ಕಾರ್ಯ	ಏನು ಮಾಡುತ್ತೇವೆ	ದೈನಂದಿನ ಹೋಲಿಕೆ
೧	ಕಾಣೆಯಾದ ಎತ್ತರವನ್ನು ಅಂದಾಜು ಮಾಡಿ ತುಂಬುವುದು	ಹಾಜರಾತಿ ಪುಸ್ತಕದಲ್ಲಿ ಒಬ್ಬನ ಹೆಸರು ಬಿಟ್ಟುಹೋದರೆ, ಅವನ ತರಗತಿಯ ಹುಡುಗರ ಸರಾಸರಿಯಿಂದ ಅಂದಾಜಿಸುವುದು
೨–೪	ತಪ್ಪಾಗಿ ಬರೆದ (ಅಸಹಜ) ಮೌಲ್ಯಗಳನ್ನು ಹುಡುಕುವುದು	ಮಕ್ಕಳ ಪಟ್ಟಿಯಲ್ಲಿ “ವಯಸ್ಸು 88” ಎಂದಿದ್ದರೆ ಅದು ಟೈಪೋ, ಡೇಟಾ ಅಲ್ಲ
೫ (ಮೊದಲ ಭಾಗ)	ಎತ್ತರ (cm) ಮತ್ತು ತೂಕ (kg) ಎರಡನ್ನೂ 0–1 ಅಳತೆಪಟ್ಟಿಗೆ ತರುವುದು	ಎರಡು ವಿದ್ಯಾರ್ಥಿಗಳ ಅಂಕಗಳನ್ನು (100ರಲ್ಲಿ ಮತ್ತು 20ರಲ್ಲಿ) ಶೇಕಡಾಕ್ಕೆ ಪರಿವರ್ತಿಸಿ ಹೋಲಿಸುವುದು
೫	ಎತ್ತರ + ತೂಕದಿಂದ ಲಿಂಗವನ್ನು ಊಹಿಸುವ ಸಣ್ಣ ನೆಟ್‌ವರ್ಕ್ (ನಾವೇ ಬರೆದ ಕೋಡ್)	ಅನುಭವಿ ಟೈಲರ್ ಅಳತೆ ನೋಡಿ ಯಾರ ಬಟ್ಟೆ ಎಂದು ಊಹಿಸುವಂತೆ
೬	70,000 ಕೈಬರಹದ ಅಂಕೆಗಳು → 10 ವರ್ಗ	ಅಂಚೆ ಕಚೇರಿಯಲ್ಲಿ ಪಿನ್‌ಕೋಡ್ ಓದುವ ಯಂತ್ರ
೬.೧–೬.೩	ಪದರಗಳು, ಆಕ್ಟಿವೇಶನ್, ಆಪ್ಟಿಮೈಜರ್ ಬದಲಿಸಿ ಹೋಲಿಸುವುದು	ಅಡುಗೆ ಪ್ರಯೋಗ — ಒಮ್ಮೆಗೆ ಒಂದೇ ಪದಾರ್ಥ ಬದಲಿಸಿ ರುಚಿ ನೋಡುವುದು

ಇಡೀ ಲ್ಯಾಬ್ ಹಿಡಿದಿಡುವ ನಿಯಮ

“Garbage in, garbage out” — ಕಸ ಒಳಗೆ ಹಾಕಿದರೆ ಕಸವೇ ಹೊರಗೆ ಬರುತ್ತದೆ. ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ನೀವು ತೋರಿಸಿದ್ದನ್ನೇ ಕಲಿಯುತ್ತದೆ — ಟೈಪೋ ಇದ್ದರೂ, ತಪ್ಪು ಅಳತೆ ಇದ್ದರೂ. ಅದಕ್ಕೇ ಕಾರ್ಯ ೧–೪ ಮಾಡೆಲನ್ನು ಕೆಟ್ಟ ಡೇಟಾದಿಂದ ಕಾಪಾಡುತ್ತವೆ; ಕಾರ್ಯ ೫–೬ ಮಾಡೆಲ್‌ಗೆ ಕಲಿಸುತ್ತವೆ; ೬.೧–೬.೩ ಯಾವ ಆಯ್ಕೆಗಳು ನಿಜವಾಗಿ ಮುಖ್ಯ ಎಂದು ಸಾಬೀತು ಮಾಡುತ್ತವೆ. ಮತ್ತು ಯಾವಾಗಲೂ, ಮಾಡೆಲ್ **ಹಿಂದೆಂದೂ ನೋಡದ ಡೇಟಾ** (test set) ಮೇಲೆ ಅಳೆಯಿರಿ.

೧. ನಿಜ ಜೀವನದಲ್ಲಿ ಇದು ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?

ಲ್ಯಾಬ್‌ನ ಉದಾಹರಣೆಗಳು ಚಿಕ್ಕವು. ಆದರೆ ಪ್ರತಿಯೊಂದೂ ಕಂಪನಿಗಳು ದಿನಾ ಮಾಡುವ ಕೆಲಸದ ಚಿಕ್ಕ ರೂಪ.

ದೃಶ್ಯ ೧: ಆಸ್ಪತ್ರೆಯ ದಾಖಲಾತಿ ವ್ಯವಸ್ಥೆ (ಕಾರ್ಯ ೧ ಮತ್ತು ೫)

ಒಬ್ಬ ನರ್ಸ್ ಒಬ್ಬ ರೋಗಿಯ ಎತ್ತರ ನಮೂದಿಸಲು ಮರೆತರು. ಸಿಸ್ಟಮ್ ಕ್ರ್ಯಾಶ್ ಆಗಬಾರದು; "0 cm" ಎಂದೂ ಹಾಕಬಾರದು — ಏಕೆಂದರೆ 0 ಹಾಕಿದರೆ ಸರಾಸರಿ ಮತ್ತು ಔಷಧ-ಪ್ರಮಾಣದ ಲೆಕ್ಕಗಳೆಲ್ಲ ಹಾಳು. ಹಾಗಾಗಿ ಅದೇ ಲಿಂಗದ, ಅದೇ ತರಹದ ರೋಗಿಗಳ ಆಧಾರದ ಮೇಲೆ ಒಂದು ಸಮಂಜಸ ಮೌಲ್ಯ ತುಂಬಿ, ಅದರ ಪಕ್ಕ "ಅಂದಾಜು" ಎಂಬ ಗುರುತು ಹಾಕುತ್ತಾರೆ. ನಂತರ ಒಂದು ಮಾಡೆಲ್ ಎತ್ತರ + ತೂಕ ಬಳಸಿ ಏನನ್ನೋ ಊಹಿಸುತ್ತದೆ (ಲ್ಯಾಬ್‌ನಲ್ಲಿ ಲಿಂಗ; ನಿಜ ಜೀವನದಲ್ಲಿ ಅಪಾಯದ ಅಂಕ, ಔಷಧ ಪ್ರಮಾಣದ ವ್ಯಾಪ್ತಿ).

ಲಿಂಗ ಯಾಕೆ ಮುಖ್ಯ?

ಈ ಡೇಟಾದಲ್ಲಿ ಮಹಿಳೆಯರ ಸರಾಸರಿ ಎತ್ತರ ≈ 166 cm, ಪುರುಷರದು ≈ 183 cm. ಎಲ್ಲರ ಸರಾಸರಿ (≈ 176) ಬಳಸಿದರೆ ಕಾಣೆಯಾದ ಮಹಿಳೆಯ ಎತ್ತರ ಸುಮಾರು 10 cm ಜಾಸ್ತಿ ಬರುತ್ತದೆ. ಒಳ್ಳೆಯ ಇಂಪ್ಯೂಟೇಶನ್ ಯಾವಾಗಲೂ "ಆ ವ್ಯಕ್ತಿ ಸೇರಿದ ಗುಂಪಿನ" ಮೌಲ್ಯ ಬಳಸುತ್ತದೆ.

ದೃಶ್ಯ ೨: ಕ್ರೀಡಾ ನೋಂದಣಿ (ಕಾರ್ಯ ೨–೪)

ಶಾಲೆಗಳು ಸ್ಪರ್ಧೆಗೆ ಮಕ್ಕಳ ಪಟ್ಟಿ ಕಳುಹಿಸುತ್ತವೆ. ಒಂದು ಪಟ್ಟಿಯಲ್ಲಿ ಒಬ್ಬನ ವಯಸ್ಸು "88" ಎಂದಿದೆ. 30 ಜನರ ಪಟ್ಟಿಯನ್ನು ಮನುಷ್ಯ ಕಣ್ಣಲ್ಲೇ ನೋಡಬಹುದು; ಆದರೆ 30 ಲಕ್ಷ ದಾಖಲೆಗಳನ್ನು ಯಾರೂ ನೋಡಲಾರರು. ಆದ್ದರಿಂದ ಸಾಫ್ಟ್‌ವೇರ್ "ಅಂಕಿ-ಅಂಶದ ಪ್ರಕಾರ ತುಂಬಾ ದೂರ ಇರುವ" ಮೌಲ್ಯಗಳನ್ನು ಗುರುತಿಸುತ್ತದೆ. ಮೂರು ಬೇರೆ ನಿಯಮಗಳು ಏಕೆ ಇವೆ ಎಂದರೆ — ಪ್ರತಿಯೊಂದೂ ಬೇರೆ ಬೇರೆ ಸನ್ನಿವೇಶದಲ್ಲಿ "ಕುರುಡು".

ಎಲ್ಲೆಡೆ ಇದೇ ಕಲ್ಪನೆ

ಕ್ರೆಡಿಟ್ ಕಾರ್ಡ್ ವಂಚನೆ (ದಿನಕ್ಕೆ ₹400 ಖರ್ಚು ಮಾಡುವ ಕಾರ್ಡ್‌ನಿಂದ ಒಮ್ಮೆಲೇ ₹9 ಲಕ್ಷದ ಖರೀದಿ), ಕಾರ್ಖಾನೆಯ ಸೆನ್ಸರ್ ಕೆಟ್ಟಿರುವುದು (ತಾಪಮಾನ 900 °C ಎಂದು ತೋರಿಸುವುದು), ಸರ್ವರ್ ದಾಳಿ (ಒಂದು ನಿಮಿಷದಲ್ಲಿ 10,000 ಲಾಗಿನ್ ಪ್ರಯತ್ನ).

ದೃಶ್ಯ ೩: ಅಂಚೆ ಕಚೇರಿ / ಬ್ಯಾಂಕ್‌ನಲ್ಲಿ ಅಂಕೆ ಓದುವುದು (ಕಾರ್ಯ ೬)

ಲಕೋಟೆಯ ಮೇಲಿನ ಪಿನ್‌ಕೋಡ್ ಮತ್ತು ಚೆಕ್‌ನ ಮೊತ್ತ ಓದುವುದು ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ಗಳ ಮೊದಲ ದೊಡ್ಡ ಯಶಸ್ಸುಗಳಲ್ಲಿ ಒಂದು. ಕ್ಯಾಮೆರಾ ಚಿತ್ರ ತೆಗೆಯುತ್ತದೆ → ಚಿತ್ರವನ್ನು ಅಂಕೆಗಳಾಗಿ ಕತ್ತರಿಸುತ್ತದೆ → ಪ್ರತಿ ಅಂಕೆ 28×28 ಸಂಖ್ಯೆಗಳ ಜಾಲರಿ ಆಗುತ್ತದೆ → ನೆಟ್‌ವರ್ಕ್ "ಇದು 7, 98% ಖಚಿತತೆಯಿಂದ" ಎನ್ನುತ್ತದೆ. ಖಚಿತತೆ ಕಡಿಮೆ ಇದ್ದರೆ ಮನುಷ್ಯ ಪರಿಶೀಲಿಸುತ್ತಾನೆ.

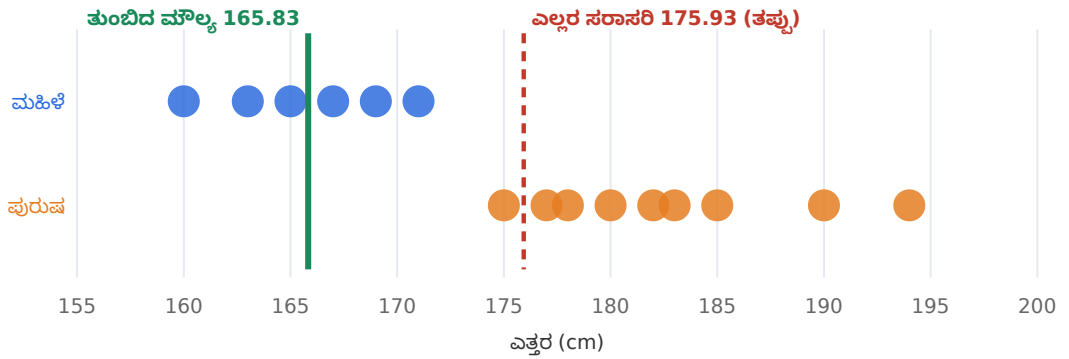
MNIST ಎಂದರೆ ಮೆಷಿನ್ ಲರ್ನಿಂಗ್‌ನ "Hello World"

784 ಸಂಖ್ಯೆಗಳು ಹೇಗೆ 10 ಉತ್ತರಗಳಲ್ಲಿ ಒಂದಾಗುತ್ತವೆ ಎಂದು ಅರ್ಥವಾದರೆ, ಚಿತ್ರ ವರ್ಗೀಕರಣ, ಧ್ವನಿ ಗುರುತಿಸುವಿಕೆ, ಪಠ್ಯ ವರ್ಗೀಕರಣ — ಎಲ್ಲದರ ಅಸ್ಥಿಪಂಜರ ನಿಮಗೆ ಅರ್ಥವಾದಂತೆ.

೨. ಕಾರ್ಯ ೧ — ಕಾಣೆಯಾದ ಮೌಲ್ಯ ತುಂಬುವುದು (Imputation)

> **ಪರಿಕಲ್ಪನೆ**
>
> **Imputation** = ಕಾಣೆಯಾದ ಮೌಲ್ಯದ ಬದಲು ಒಂದು ಒಳ್ಳೆಯ ಅಂದಾಜು ಹಾಕುವುದು. **Mean (ಸರಾಸರಿ)** = ಎಲ್ಲವನ್ನು ಕೂಡಿ ಸಂಖ್ಯೆಯಿಂದ ಭಾಗಿಸಿದ್ದು. **Median (ಮಧ್ಯಸ್ಥ)** = ಸಾಲಾಗಿ ಜೋಡಿಸಿದಾಗ ನಡುವೆ ಬರುವ ಮೌಲ್ಯ — ತೀರಾ ದೊಡ್ಡ/ಚಿಕ್ಕ ಮೌಲ್ಯಗಳು ಇದನ್ನು ಅಲುಗಾಡಿಸುವುದಿಲ್ಲ.

ಲ್ಯಾಬ್‌ನಲ್ಲಿ 16 ಜನರ ಪಟ್ಟಿ ಇದೆ (gender 0 = ಮಹಿಳೆ, 1 = ಪುರುಷ). ಹನ್ನೊಂದನೇ ವ್ಯಕ್ತಿ (Name 11, ಮಹಿಳೆ) ಎತ್ತರ ಕಾಣೆಯಾಗಿದೆ.



ಚಿತ್ರ ೨.೧: ಮಹಿಳೆಯರ (ನೀಲಿ) ಮತ್ತು ಪುರುಷರ (ಕಿತ್ತಳೆ) ಎತ್ತರಗಳು. ಹಸಿರು ಗೆರೆ = ಮಹಿಳೆಯರ ಸರಾಸರಿಯಿಂದ ತುಂಬಿದ ಮೌಲ್ಯ; ಕೆಂಪು ಚುಕ್ಕೆ ಗೆರೆ = “ಎಲ್ಲರ” ಸರಾಸರಿ (ತಪ್ಪು ದಾರಿ)

ಹಂತ-ಹಂತದ ಲೆಕ್ಕ

ಹಂತ	ಲೆಕ್ಕ	ಫಲಿತಾಂಶ
೧. ಮಹಿಳೆಯರ (ಕಾಣೆಯಾದವರನ್ನು ಬಿಟ್ಟು) ಎತ್ತರಗಳು	165, 171, 160, 169, 167, 163	6 ಮೌಲ್ಯಗಳು
೨. ಮೊತ್ತ	165+171+160+169+167+163	995
೩. **Mean**	995 ÷ 6	**165.83 cm**
೪. **Median** — ಜೋಡಿಸಿ: 160, 163, **165, 167**, 169, 171	ನಡುವಿನ ಎರಡರ ಸರಾಸರಿ (165+167)÷2	**166 cm**
೫. ಪುರುಷರ ಸರಾಸರಿ (9 ಜನ)	1644 ÷ 9	182.67 cm
೬. ಎಲ್ಲರ ಸರಾಸರಿ (15 ಜನ)	(995+1644) ÷ 15	175.93 cm — ತಪ್ಪು ದಾರಿ!

ನಿಜ ಜೀವನದ ಹೋಲಿಕೆ

ನಿಮ್ಮ ತರಗತಿಯಲ್ಲಿ ಒಬ್ಬ ಹುಡುಗಿ ಪರೀಕ್ಷೆಗೆ ಬಂದಿಲ್ಲ; ಅವಳ ಎತ್ತರ ಗೊತ್ತಿಲ್ಲ. ನೀವು ಇಡೀ ಶಾಲೆಯ (ಪ್ರಾಧ್ಯಾಪಕರು ಸೇರಿ) ಸರಾಸರಿ ತೆಗೆದುಕೊಳ್ಳುತ್ತೀರಾ, ಅಥವಾ ತರಗತಿಯ ಹುಡುಗಿಯರ ಸರಾಸರಿಯನ್ನೇ? ಎರಡನೆಯದೇ ಸರಿ. ಅಷ್ಟೇ ಇಲ್ಲಿಯೂ.

ಲ್ಯಾಬ್‌ನಲ್ಲಿ ಏನಾಯಿತು?

Mean ಬಳಸಿದಾಗ **165.83**, median ಬಳಸಿದಾಗ **166** — ಎರಡೂ ಬಹುತೇಕ ಒಂದೇ. ಆದರೆ ಡೇಟಾದಲ್ಲಿ ವಿಪರೀತ ಮೌಲ್ಯ ಇದ್ದರೆ median ಸುರಕ್ಷಿತ. “Everyone” ಆಯ್ಕೆ ಮಾಡಿದರೆ ತುಂಬುವ ಮೌಲ್ಯ ಮಹಿಳೆಯರ ಗುಂಪಿನಿಂದ ದೂರ ಹೋಗುತ್ತದೆ — ಅದೇ ನಾವು ತರುವ ತಪ್ಪು.

೩. ಕಾರ್ಯ ೨–೪ — ಅಸಹಜ ಮೌಲ್ಯ ಪತ್ತೆ

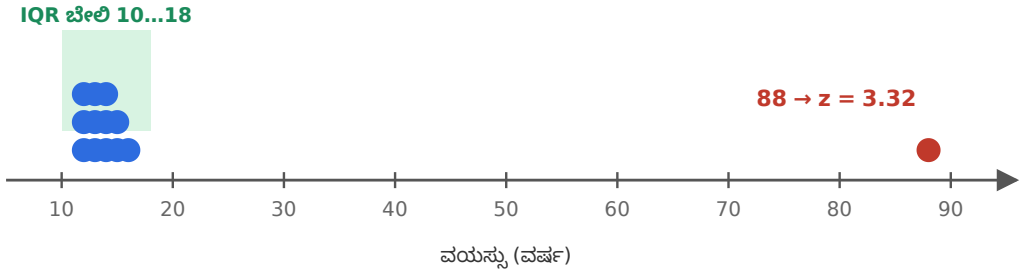
ಪರಿಕಲ್ಪನೆ

Outlier / Anomaly = ಉಳಿದ ಡೇಟಾಗೆ ಹೊಂದದ ಮೌಲ್ಯ. ಲ್ಯಾಬ್‌ನ ಮೂರು ವಿಧಾನಗಳೂ “ಎಷ್ಟು ದೂರ ಎಂದರೆ ತುಂಬಾ ದೂರ?” ಎಂಬ ಒಂದೇ ಪ್ರಶ್ನೆಗೆ ಬೇರೆ ಬೇರೆ ರೀತಿಯಲ್ಲಿ ಉತ್ತರಿಸುತ್ತವೆ.

ವಿಧಾನ	ಕೇಳುವ ಪ್ರಶ್ನೆ	ಸೂತ್ರ	ದೌರ್ಬಲ್ಯ
Z-score	“ಸರಾಸರಿಯಿಂದ ಎಷ್ಟು ಪ್ರಮಾಣಿತ ವಿಚಲನ (σ) ದೂರ?”	$z = (x - \mu) \div \sigma$; $\lvert z\rvert > 3$ ಆದರೆ ಅಸಹಜ	ಬೆಲ್-ಕರ್ವ್ (ಸಾಮಾನ್ಯ ವಿತರಣೆ) ಇರಬೇಕು; ಅಸಹಜ ಮೌಲ್ಯವೇ σ ಅನ್ನು ಉಬ್ಬಿಸಿ ತನ್ನನ್ನು ಬಚ್ಚಿಟ್ಟುಕೊಳ್ಳಬಹುದು
Chauvenet	“n ಮಾದರಿಗಳಲ್ಲಿ ಇಷ್ಟು ದೂರದ ಮೌಲ್ಯ ಆಕಸ್ಮಿಕವಾಗಿ ಎಷ್ಟು ಬರಬಹುದು?”	$n \cdot P(\lvert Z\rvert \geq z) < 0.5$ ಆದರೆ ತಿರಸ್ಕರಿಸಿ	ಇದೂ σ ಅನ್ನೇ ಅವಲಂಬಿಸಿದೆ; ಆದರೆ n ಅನ್ನು ಗಣನೆಗೆ ತೆಗೆದುಕೊಳ್ಳುತ್ತದೆ
Quartile (IQR)	“ಮಧ್ಯದ 50% ಡೇಟಾದ ಬೇಲಿಯ ಹೊರಗೆ ಇದೆಯೇ?”	$Q1 - 1.5 \cdot IQR \ldots Q3 + 1.5 \cdot IQR$	ಕಡಿಮೆ — ಏಕೆಂದರೆ ಪರ್ಸೆಂಟೈಲ್ ಬಳಸುತ್ತದೆ, ಅಸಹಜ ಮೌಲ್ಯ ಇದನ್ನು ಬಾಗಿಸಲಾರದು

ಸಣ್ಣ ಉದಾಹರಣೆಯಲ್ಲಿ ಪೂರ್ಣ ಲೆಕ್ಕ

ಲ್ಯಾಬ್‌ನ 30 ಮಕ್ಕಳ ಪಟ್ಟಿ ಇಲ್ಲಿ ಇಲ್ಲದ ಕಾರಣ, ಅದೇ ಕಲ್ಪನೆಯ 13 ಮಕ್ಕಳ ಒಂದು ಚಿಕ್ಕ ಪಟ್ಟಿ ತೆಗೆದುಕೊಳ್ಳೋಣ: `12, 13, 14, 12, 15, 13, 14, 16, 13, 12, 14, 15, 88`. ಕೊನೆಯ “88” ಸ್ಪಷ್ಟವಾಗಿ ಟೈಪೋ.



ಚಿತ್ರ ೩.೧: 88 ಉಳಿದವರಿಂದ ಬಹಳ ದೂರ ಇದೆ. ಹಸಿರು ಪ್ರದೇಶ = IQR ಬೇಲಿಯ ಒಳಗಿನ ಸುರಕ್ಷಿತ ವ್ಯಾಪ್ತಿ (10 … 18)

ಹಂತ	Z-score	Chauvenet	IQR
ಬೇಕಾದ ಅಂಕಿ	μ = 19.31, σ = 20.68	n = 13, z = 3.32	Q1 = 13, Q3 = 15, IQR = 2
ಲೆಕ್ಕ	z(88) = (88 − 19.31) ÷ 20.68 = **3.32**	13 × P(|Z| ≥ 3.32) = 13 × 0.0009 = **0.0116**	ಮೇಲಿನ ಬೇಲಿ = 15 + 1.5×2 = **18**; ಕೆಳಗಿನ = 13 − 3 = **10**
ತೀರ್ಮಾನ	3.32 > 3 → **ಅಸಹಜ**	0.0116 < 0.5 → **ತಿರಸ್ಕೃತ**	88 > 18 → **ಅಸಹಜ**

ಗಮನಿಸಿ: 88 ಅನ್ನು ತೆಗೆದರೆ ಉಳಿದ 12 ಮಕ್ಕಳ σ ಕೇವಲ **1.31** — ಆದರೆ 88 ಇರುವಾಗ σ = 20.68! ಒಂದು ತಪ್ಪು ಮೌಲ್ಯ σ ಅನ್ನು ಹದಿನಾರು ಪಟ್ಟು ಉಬ್ಬಿಸಿತು.

ಅಡಗಿಕೆ (Masking) — Z-score ಏಕೆ ಒಮ್ಮೊಮ್ಮೆ ಸೋಲುತ್ತದೆ?

> **ಒಂದು ಕಳ್ಳನ ಕಥೆ**
>
> ಒಬ್ಬ ಕಳ್ಳ ತಾನು ತಪ್ಪಿಸಿಕೊಳ್ಳಲು ತಾನೇ ಇರುವ ಗುಂಪಿನಲ್ಲಿ ಗದ್ದಲ ಎಬ್ಬಿಸಿದರೆ ಪೊಲೀಸರಿಗೆ ಯಾರು ಕಳ್ಳ ಎಂದು ತಿಳಿಯುವುದು ಕಷ್ಟ. ಹಾಗೆಯೇ ಅಸಹಜ ಮೌಲ್ಯ ತನ್ನದೇ ಪ್ರಭಾವದಿಂದ σ ಅನ್ನು ದೊಡ್ಡದು ಮಾಡಿ, ತನ್ನ z-score ಅನ್ನು ತಾನೇ ಚಿಕ್ಕದು ಮಾಡಿಕೊಳ್ಳುತ್ತದೆ.

ಇನ್ನೊಂದು ಪಟ್ಟಿ (10 ಮಕ್ಕಳು): `12, 13, 14, 12, 15, 13, 14, 16, 13, 40` — 40 ಕೂಡ ಸ್ಪಷ್ಟವಾಗಿ ತಪ್ಪು.

ವಿಧಾನ	ಲೆಕ್ಕ	ತೀರ್ಮಾನ
Z-score	μ = 16.2, σ = 8.46 → z = (40−16.2)÷8.46 = **2.81**	**3ಕ್ಕಿಂತ ಕಡಿಮೆ → ಸಿಕ್ಕಿಬೀಳಲಿಲ್ಲ!**
Chauvenet	10 × P(|Z| ≥ 2.81) = 10 × 0.0049 = **0.049**	**0.5ಕ್ಕಿಂತ ಬಹಳ ಕಡಿಮೆ → ಅಸಹಜ**
IQR	Q1 = 13, Q3 = 14.75, IQR = 1.75 → ಮೇಲಿನ ಬೇಲಿ = 14.75 + 2.625 = **17.375**	**40 > 17.375 → ಅಸಹಜ**

ಲ್ಯಾಬ್‌ನ ಅದೇ ಪ್ರಯೋಗ

ಮೂಲ ಗೈಡ್‌ನ ಲೈವ್ ಡೆಮೊದಲ್ಲಿ 30 ಮಕ್ಕಳ ಪಟ್ಟಿಯ "88" ಅನ್ನು 30–40ಕ್ಕೆ ಇಳಿಸಿದರೆ Z-score (ಮಿತಿ 3) ಅದನ್ನು ಹಿಡಿಯುವುದಿಲ್ಲ, ಆದರೆ quartile ನಿಯಮ ಹಿಡಿಯುತ್ತದೆ. ಅದಕ್ಕೇ **ಒಂದೇ ವಿಧಾನದ ಮೇಲೆ ನಂಬಿಕೆ ಇಡಬಾರದು** — ಲ್ಯಾಬ್ ಮೂರೂ ಕೇಳುತ್ತದೆ.

ಲ್ಯಾಬ್‌ನ ನಿಜವಾದ ಫಲಿತಾಂಶ (30 ಮಕ್ಕಳು, ಮೌಲ್ಯ 88)

mean = **14.73**, std = **14.04**, z(88) = **5.22**; Chauvenet n·P = **5.4×10⁻⁶** (0.5ಕ್ಕಿಂತ ತುಂಬಾ ಕಡಿಮೆ); quartile ಮಿತಿಗಳು **4.625 … 19.625**. ಮೂರೂ ವಿಧಾನಗಳು 88 ಅನ್ನು ಮಾತ್ರ ಅಸಹಜ ಎಂದು ತೋರಿಸಿದವು. 88 ಇಲ್ಲದೆ ನಿಜವಾದ ಹರಡುವಿಕೆ ಕೇವಲ ≈ 2.4 — ಅಂದರೆ ಆ ಒಂದು ಮೌಲ್ಯವೇ std ಅನ್ನು ಸುಮಾರು ಆರು ಪಟ್ಟು ದೊಡ್ಡದಾಗಿಸಿತ್ತು.

ಅಸಹಜ ಮೌಲ್ಯ ಸಿಕ್ಕಮೇಲೆ ಏನು ಮಾಡಬೇಕು?

ಯಾವಾಗಲೂ ಅಳಿಸಬೇಡಿ! ಮೊದಲು ಕೇಳಿ: ಇದು ಟೈಪೋ ಆಗಿದ್ದರೆ ಸರಿಪಡಿಸಿ (88 → 8 ಅಥವಾ 18?), ಸೆನ್ಸರ್ ದೋಷವಾದರೆ ತೆಗೆಯಿರಿ, ನಿಜವಾದ ಅಪರೂಪದ ಘಟನೆ (ಉದಾ: ನಿಜವಾದ ವಂಚನೆ) ಆದರೆ ಇಟ್ಟುಕೊಂಡು ಪ್ರತ್ಯೇಕವಾಗಿ ಅಧ್ಯಯನ ಮಾಡಿ.

೪. ಸಾಮಾನ್ಯೀಕರಣ (Normalization) — ಎಲ್ಲರಿಗೂ ಒಂದೇ ಅಳತೆಪಟ್ಟಿ

ಎತ್ತರ ಸುಮಾರು 150–200 (cm), ತೂಕ ಸುಮಾರು 50–100 (kg). ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಸಣ್ಣ ಯಾದೃಚ್ಛಿಕ ತೂಕಗಳಿಂದ ಶುರುವಾಗಿ ಸಣ್ಣ ಸಣ್ಣ ಹೆಜ್ಜೆಗಳಲ್ಲಿ ಕಲಿಯುತ್ತದೆ. ದೊಡ್ಡ ಸಂಖ್ಯೆಯ ಫೀಚರ್ ಉಳಿದವನ್ನು ಅಡಗಿಸಿ ಬಿಡುತ್ತದೆ; ಗ್ರೇಡಿಯೆಂಟ್‌ಗಳೂ ಅಸ್ಥಿರವಾಗುತ್ತವೆ. ಅದಕ್ಕೇ ಎಲ್ಲವನ್ನೂ ಒಂದೇ ಮಟ್ಟಕ್ಕೆ ತರುತ್ತೇವೆ.

ದೈನಂದಿನ ಉದಾಹರಣೆ

ಒಬ್ಬ ವಿದ್ಯಾರ್ಥಿ ಗಣಿತದಲ್ಲಿ 100ರಲ್ಲಿ 80, ಇನ್ನೊಬ್ಬ ಪ್ರಾಜೆಕ್ಟ್‌ನಲ್ಲಿ 20ರಲ್ಲಿ 18. ಮೊದಲನೆಯದು "ದೊಡ್ಡ ಸಂಖ್ಯೆ" ಎಂದು ಹೋಲಿಸಿದರೆ ಅನ್ಯಾಯ. ಇಬ್ಬರನ್ನೂ ಶೇಕಡಾಕ್ಕೆ ತಂದರೆ (80% ಮತ್ತು 90%) ನ್ಯಾಯಯುತ ಹೋಲಿಕೆ ಸಾಧ್ಯ. ಇದೇ min-max ಸ್ಕೇಲಿಂಗ್.

	Min-max → [0, 1]	Standardize (ಸರಾಸರಿ 0, std 1)
ಸೂತ್ರ	$x' = (x - \min) \div (\max - \min)$	$x' = (x - \mu) \div \sigma$
ಫಲಿತಾಂಶದ ಹರವು	ಯಾವಾಗಲೂ 0 ರಿಂದ 1	ಬಂಧಿತವಲ್ಲ (ಋಣ ಸಂಖ್ಯೆಗಳೂ ಬರುತ್ತವೆ)
ದೌರ್ಬಲ್ಯ	ಅಸಹಜ ಮೌಲ್ಯಕ್ಕೆ ಸೂಕ್ಷ್ಮ — ಅದಕ್ಕೇ ಕಾರ್ಯ ೨–೪ ಮೊದಲು!	σ ಮೇಲೆ ಅವಲಂಬನೆ
ಲ್ಯಾಬ್‌ನಲ್ಲಿ	ಕಾರ್ಯ ೫ ರಲ್ಲಿ ಬಳಕೆ	ಉದ್ದೇಶದಲ್ಲಿ ಹೆಸರಿಸಿರುವ ಇನ್ನೊಂದು ವಿಧಾನ

ಉದಾಹರಣೆ: ಐದು ಎತ್ತರಗಳು 150, 160, 170, 180, 190

ಎತ್ತರ	Min-max: (x−150)÷40	Standardize: (x−170)÷14.14
150	0.00	−1.41
160	0.25	−0.71
170	0.50	0.00
180	0.75	0.71
190	1.00	1.41

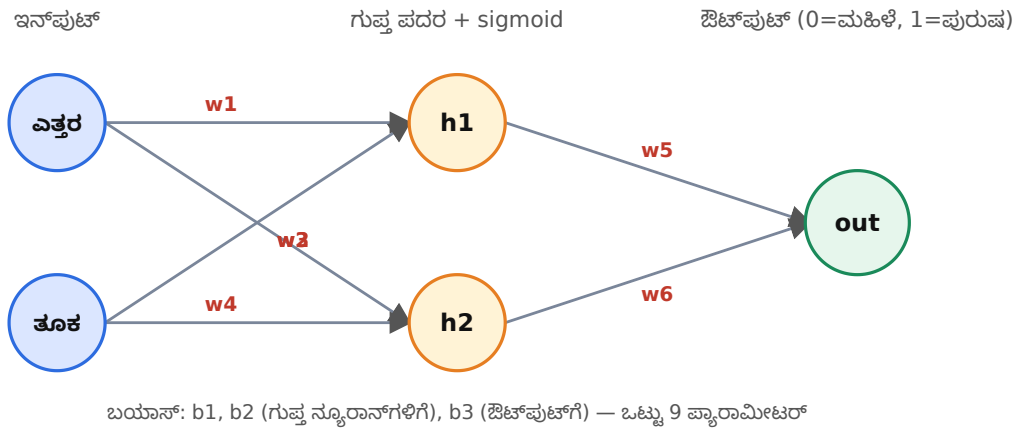
ಅತ್ಯಂತ ಮುಖ್ಯ ವಿವರ

min/max (ಅಥವಾ mean/std) ಅನ್ನು **ಟ್ರೈನಿಂಗ್ ಡೇಟಾದಿಂದಲೇ** ಲೆಕ್ಕ ಹಾಕಿ, ಹೊಸ ಡೇಟಾಗೂ ಅದೇ ಸಂಖ್ಯೆಗಳನ್ನು ಬಳಸಿ. ಟೆಸ್ಟ್ ಡೇಟಾದ ಮೇಲೆ ಮತ್ತೆ ಲೆಕ್ಕ ಹಾಕಬೇಡಿ — ಹಾಗೆ ಮಾಡಿದರೆ ಪರೀಕ್ಷೆಯ ಪ್ರಶ್ನೆಪತ್ರಿಕೆ ಮೊದಲೇ ನೋಡಿದಂತೆ (data leakage).

೫. ಕಾರ್ಯ ೫ — ಕೈಯಿಂದ ಬರೆದ ಸಣ್ಣ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್

ಪರಿಕಲ್ಪನೆ

ಒಂದು **ನ್ಯೂರಾನ್** ಪ್ರತಿ ಇನ್‌ಪುಟ್ ಅನ್ನು ಒಂದು **ತೂಕ (weight)** ದಿಂದ ಗುಣಿಸಿ, **ಬಯಾಸ್** ಕೂಡಿ, ಆ ಮೊತ್ತವನ್ನು **ಆಕ್ಟಿವೇಶನ್** ಮೂಲಕ "ಅದುಮುತ್ತದೆ". ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಎಂದರೆ ಇಂತಹ ನ್ಯೂರಾನ್‌ಗಳನ್ನು ಪದರ ಪದರವಾಗಿ ಜೋಡಿಸಿದ್ದು. ಇಲ್ಲಿನ ನೆಟ್‌ವರ್ಕ್: **2 → 2 → 1** (ಎರಡು ಇನ್‌ಪುಟ್: ಎತ್ತರ, ತೂಕ; ಎರಡು ಗುಪ್ತ ನ್ಯೂರಾನ್; ಒಂದು ಔಟ್‌ಪುಟ್).



ಚಿತ್ರ ೫.೧: 2 → 2 → 1 ನೆಟ್‌ವರ್ಕ್. ಎಲ್ಲ ಬಾಣಗಳ ಮೇಲೆ ಒಂದೊಂದು ತೂಕ; ಪ್ರತಿ ನ್ಯೂರಾನ್‌ಗೆ ಒಂದು ಬಯಾಸ್

ಕಲಿಯಬೇಕಾದ ಒಟ್ಟು ಸಂಖ್ಯೆಗಳು: 4 (ಇನ್‌ಪುಟ್→ಗುಪ್ತ ತೂಕ) + 2 (ಗುಪ್ತ ಬಯಾಸ್) + 2 (ಗುಪ್ತ→ಔಟ್) + 1 (ಔಟ್‌ಪುಟ್ ಬಯಾಸ್) = **9**. ನಂತರ ಬರುವ MNIST ಮಾಡೆಲ್‌ನಲ್ಲಿ 5,75,050 ಇವೆ — ಆದರೆ ಕಲ್ಪನೆ ಇದೇ.

ಮುಂದಕ್ಕೆ ಹೋಗುವುದು (Forward pass) — ಊಹೆ ಮಾಡುವುದು

```
h1  = sigmoid(w1·ಎತ್ತರ + w2·ತೂಕ + b1)
h2  = sigmoid(w3·ಎತ್ತರ + w4·ತೂಕ + b2)
out = sigmoid(w5·h1 + w6·h2 + b3)
```

Sigmoid ಯಾವುದೇ ಸಂಖ್ಯೆಯನ್ನು 0 ರಿಂದ 1ರ ನಡುವೆ ಅದುಮುತ್ತದೆ. ಹಾಗಾಗಿ ಔಟ್‌ಪುಟ್ ಅನ್ನು "ಪುರುಷನಾಗಿರುವ ಸಂಭವನೀಯತೆ" ಎಂದು ಓದಬಹುದು (1 = ಪುರುಷ, 0 = ಮಹಿಳೆ).

ಒಂದು ಪೂರ್ಣ ಹೆಜ್ಜೆ — ಕೈಯಿಂದ ಲೆಕ್ಕ

ಸರಳತೆಗಾಗಿ ಒಬ್ಬ ವ್ಯಕ್ತಿಯನ್ನು ತೆಗೆದುಕೊಳ್ಳೋಣ: ಸ್ಕೇಲ್ ಮಾಡಿದ ಎತ್ತರ = 0.5, ತೂಕ = 0.4, ನಿಜವಾದ ಉತ್ತರ (truth) = 1 (ಪುರುಷ). ಆರಂಭದ ತೂಕಗಳು (ಉದಾಹರಣೆಗೆ ನಾನೇ ಆರಿಸಿದ್ದು): w1 = 0.5, w2 = −0.3, b1 = 0.1; w3 = −0.2, w4 = 0.4, b2 = 0; w5 = 0.6, w6 = −0.5, b3 = 0.2. ಕಲಿಕೆಯ ವೇಗ lr = 0.5.

ಹಂತ	ಲೆಕ್ಕ	ಫಲಿತಾಂಶ
೧. h1	sigmoid(0.5×0.5 + (−0.3)×0.4 + 0.1) = sigmoid(0.23)	0.5572
೨. h2	sigmoid((−0.2)×0.5 + 0.4×0.4 + 0) = sigmoid(0.06)	0.5150
೩. out	sigmoid(0.6×0.5572 + (−0.5)×0.5150 + 0.2) = sigmoid(0.2768)	**0.5688**
೪. ದೋಷ	out − truth = 0.5688 − 1	−0.4312
೫. d_out	ದೋಷ × out × (1−out) = −0.4312 × 0.5688 × 0.4312	−0.1058
೬. d_h1	d_out × w5 × h1 × (1−h1) = −0.1058 × 0.6 × 0.5572 × 0.4428	−0.01566
೭. d_h2	d_out × w6 × h2 × (1−h2) = −0.1058 × (−0.5) × 0.5150 × 0.4850	+0.01321

ತೂಕ ಬದಲಿಸುವುದು: ಹೊಸ ತೂಕ = ಹಳೆಯದು − lr × d × (ಆ ತೂಕದ ಇನ್‌ಪುಟ್)

ತೂಕ	ಲೆಕ್ಕ	ಹಳೆಯದು → ಹೊಸದು
w5	0.6 − 0.5 × (−0.1058) × 0.5572	0.6000 → **0.6295**
w6	−0.5 − 0.5 × (−0.1058) × 0.5150	−0.5000 → **−0.4728**
b3	0.2 − 0.5 × (−0.1058)	0.2000 → **0.2529**
w1	0.5 − 0.5 × (−0.01566) × 0.5	0.5000 → **0.5039**
w2	−0.3 − 0.5 × (−0.01566) × 0.4	−0.3000 → **−0.2969**
b1	0.1 − 0.5 × (−0.01566)	0.1000 → **0.1078**

(w3, w4, b2 ಕೂಡ ಅದೇ ರೀತಿ d_h2 ಬಳಸಿ ಬದಲಾಗುತ್ತವೆ.) **ಒಂದೇ ಹೆಜ್ಜೆಯ ನಂತರ** ಮತ್ತೆ ಊಹಿಸಿದಾಗ: out = **0.5898**; ಸತ್ಯ 1ಕ್ಕೆ ದೂರ 0.4312 → **0.4102** ಗೆ ಇಳಿಯಿತು. ಇಂತಹ ಹೆಜ್ಜೆಗಳನ್ನು ಸಾವಿರಾರು ಬಾರಿ

ಪುನರಾವರ್ತಿಸುವುದೇ ತರಬೇತಿ (training).

> **ಬ್ಯಾಕ್‌ಪ್ರೊಪಗೇಶನ್ — ಒಂದು ಕಂಪನಿಯ ಉದಾಹರಣೆ**
> ಒಂದು ಹೋಟೆಲ್‌ನಲ್ಲಿ ಗ್ರಾಹಕ "ಊಟ ಚೆನ್ನಾಗಿಲ್ಲ" ಎಂದು ದೂರು ಕೊಟ್ಟರು. ಮ್ಯಾನೇಜರ್ ಹಿಂದಕ್ಕೆ ಹೋಗಿ ಕೇಳುತ್ತಾನೆ: ಸರ್ವರ್ ಎಷ್ಟು ತಪ್ಪು ಮಾಡಿದ? ಅಡುಗೆಯವ? ತರಕಾರಿ ಸರಬರಾಜುದಾರ? ಪ್ರತಿಯೊಬ್ಬರ ಪಾತ್ರದ ಪ್ರಮಾಣಕ್ಕೆ ತಕ್ಕಂತೆ ಹೊಣೆ ಹಂಚುತ್ತಾನೆ. ಬ್ಯಾಕ್‌ಪ್ರೊಪಗೇಶನ್ ಕೂಡ ಅಷ್ಟೇ — ಅಂತಿಮ ದೋಷವನ್ನು ಹಿಂದಿನ ಪದರಗಳಿಗೆ ಕಲನಶಾಸ್ತ್ರದ chain rule ಮೂಲಕ ಹಂಚುತ್ತದೆ, ಪ್ರತಿ ತೂಕಕ್ಕೂ "ಯಾವ ದಿಕ್ಕಿಗೆ ಎಷ್ಟು ಬದಲಾಗಬೇಕು" ಎಂದು ತಿಳಿಯುತ್ತದೆ.

ಲ್ಯಾಬ್‌ನ ಫಲಿತಾಂಶ — Name 5 (158 cm, 54 kg) ಯಾರು?

Run	lr	Epochs	ಅಂತಿಮ MSE	Train acc	Name 5 ಔಟ್‌ಪುಟ್	ಉತ್ತರ
1	0.01	100	0.25548	0.533	0.5611	male ✗
2	0.01	1000	0.18746	0.867	0.4465	female
3	0.1	100	0.18677	0.867	0.4469	female
4	**0.1**	**1000**	**0.00481**	**1**	**0.0248**	**female**
5	0.5	100	0.01392	1	0.0582	female
6	0.5	1000	0.00052	1	0.0051	female
7	1.0	1000	0.00022	1	0.0029	female
8	2.0	1000	0.0001	1	0.0017	female

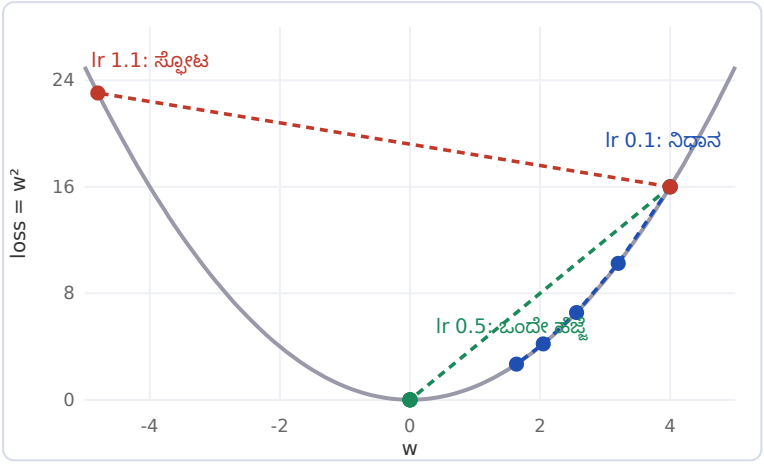
> **ತೀರ್ಮಾನ**
> lr = 0.01 ಮತ್ತು 100 epochs ಇದ್ದಾಗ ಕಲಿಕೆ ಬಹುತೇಕ ಆಗಲೇ ಇಲ್ಲ (accuracy 53%, ಉತ್ತರ ತಪ್ಪು). lr ಅಥವಾ epochs ಹೆಚ್ಚಿಸಿದರೆ ಲಾಸ್ ಕುಸಿಯುತ್ತದೆ ಮತ್ತು Name 5 ಸ್ಪಷ್ಟವಾಗಿ **ಮಹಿಳೆ (0)** ಎಂದು ಬರುತ್ತದೆ. ಅಂದರೆ: **ಸಾಕಷ್ಟು (ಕಲಿಕೆಯ ವೇಗ × ಪುನರಾವರ್ತನೆ) ಬೇಕು**.

೬. ಕಲಿಕೆಯ ವೇಗ (Learning rate) — ಬೆಟ್ಟ ಇಳಿಯುವ ಉದಾಹರಣೆ

ತರಬೇತಿ ಎಂದರೆ "ಲಾಸ್ ಬೆಟ್ಟ"ದಿಂದ ಕೆಳಗೆ ಇಳಿಯುವುದು. ಲರ್ನಿಂಗ್ ರೇಟ್ ಎಂದರೆ ಹೆಜ್ಜೆಯ ಗಾತ್ರ. ಲ್ಯಾಬ್‌ನ ಚಿಕ್ಕ ಬೆಟ್ಟ: $\text{loss} = w^2$, ಆರಂಭ $w = 4$. ಇಲ್ಲಿ ಗ್ರೇಡಿಯೆಂಟ್ $= 2w$, ಹೊಸ $w = w - \text{lr} \times 2w$.

ಕಣ್ಣುಮುಚ್ಚಿ ಬೆಟ್ಟ ಇಳಿಯುವವನ ಕಥೆ

ಮಂಜು ತುಂಬಿದ ಬೆಟ್ಟದಲ್ಲಿ ನೀವು ಕೆಳಗಿನ ಕಣಿವೆ ತಲುಪಬೇಕು. ಕಾಲಿನ ಕೆಳಗಿನ ಇಳಿಜಾರು ಮಾತ್ರ ಗೊತ್ತು (ಅದೇ gradient). ಚಿಕ್ಕ ಹೆಜ್ಜೆ ಇಟ್ಟರೆ ಸುರಕ್ಷಿತ ಆದರೆ ಬಹಳ ನಿಧಾನ. ದೊಡ್ಡ ಜಿಗಿತ ಹಾಕಿದರೆ ಕಣಿವೆಯನ್ನು ದಾಟಿ ಆ ಕಡೆಯ ಬೆಟ್ಟ ಏರುತ್ತೀರಿ.



ಚಿತ್ರ ೬.೧: loss = w^2 ಮೇಲೆ ನಾಲ್ಕು ಹೆಜ್ಜೆಗಳು — lr = 0.1 (ನಿಧಾನ), 0.5 (ಒಂದೇ ಹೆಜ್ಜೆಯಲ್ಲಿ ತಲುಪುತ್ತದೆ), 1.1 (ಸ್ಫೋಟ)

lr	w: ಹೆಜ್ಜೆ 0 → 1 → 2 → 3 → 4	ಏನಾಗುತ್ತದೆ?
0.1	4 → 3.2 → 2.56 → 2.05 → 1.64	ಸುರಕ್ಷಿತ, ಆದರೆ ನಿಧಾನ
0.5	4 → 0 → 0 → 0 → 0	ಈ ಬೆಟ್ಟಕ್ಕೆ ಪರಿಪೂರ್ಣ
1.0	4 → −4 → 4 → −4 → 4	ಕಣಿವೆ ದಾಟಿ ಅತ್ತಿತ್ತ ಜಿಗಿಯುತ್ತದೆ, ಎಂದಿಗೂ ಸ್ಥಿರವಾಗುವುದಿಲ್ಲ
1.1	4 → −4.8 → 5.76 → −6.91 → 8.29	**ಸ್ಫೋಟ (diverge) — ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ ಹೆಚ್ಚು ದೂರ**

೭. ಕಾರ್ಯ ೬ — MNIST ಅಂಕೆಗಳನ್ನು ಓದುವ ನೆಟ್‌ವರ್ಕ್

MNIST ಎಂದರೇನು?

60,000 ತರಬೇತಿ + 10,000 ಪರೀಕ್ಷಾ ಚಿತ್ರಗಳು; ಪ್ರತಿಯೊಂದು ಕೈಬರಹದ ಅಂಕೆ, 28×28 ಪಿಕ್ಸೆಲ್, ಬೂದು ಬಣ್ಣದ ಮೌಲ್ಯ 0–255. “Fully connected (Dense)” ಎಂದರೆ ಪ್ರತಿ ನ್ಯೂರಾನ್ ಹಿಂದಿನ ಪದರದ ಪ್ರತಿ ನ್ಯೂರಾನ್ ಅನ್ನೂ ನೋಡುತ್ತದೆ.

ಹಂತ ೧: ಚಿತ್ರ ಎಂದರೆ ಸಂಖ್ಯೆಗಳು ಮಾತ್ರ

ಕಂಪ್ಯೂಟರ್‌ಗೆ “7” ಒಂದು ಚಿತ್ರವಲ್ಲ — 28 ಸಾಲು × 28 ಕಂಬಗಳ ಸಂಖ್ಯೆಗಳ ಕೋಷ್ಟಕ. 0 = ಕಪ್ಪು, 255 = ಬಿಳಿ. ಅಂಕೆಯ ಗೆರೆ ಹಾದುಹೋಗುವ ಜಾಗದಲ್ಲಿ ದೊಡ್ಡ ಸಂಖ್ಯೆ, ಉಳಿದೆಡೆ 0.

ಹಂತ ೨: ಚಪ್ಪಟೆ ಮಾಡುವುದು (Flatten) ಮತ್ತು ಭಾಗಿಸುವುದು

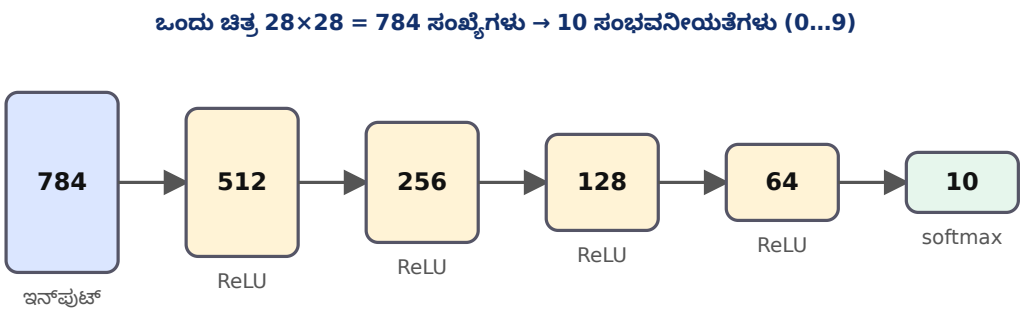
Dense ಪದರಕ್ಕೆ ಜಾಲರಿ ಬೇಡ, ಒಂದೇ ಸಾಲಿನ ಪಟ್ಟಿ ಬೇಕು. 28 ಸಾಲುಗಳನ್ನು ಒಂದರ ಹಿಂದೆ ಒಂದು ಜೋಡಿಸಿದರೆ **28 × 28 = 784** ಸಂಖ್ಯೆಗಳು. ನಂತರ ಎಲ್ಲವನ್ನೂ 255 ರಿಂದ ಭಾಗಿಸಿ 0–1 ಮಟ್ಟಕ್ಕೆ ತರುತ್ತೇವೆ (ಇದೇ ಹಿಂದಿನ ಸಾಮಾನ್ಯೀಕರಣ).

> **ಉದಾಹರಣೆ**
> ಒಂದು ಪಿಕ್ಸೆಲ್ ಮೌಲ್ಯ 191 ಆದರೆ 191 ÷ 255 = 0.749. ಕಪ್ಪು 0 → 0.0, ಬಿಳಿ 255 → 1.0.

ಹಂತ ೩: ಲೇಬಲ್‌ಗಳು One-hot ಆಗುತ್ತವೆ

ಅಂಕೆ 7 ಅನ್ನು `[0,0,0,0,0,0,0,1,0,0]` ಎಂದು ಬರೆಯುತ್ತೇವೆ. ನೆಟ್‌ವರ್ಕ್ ಹತ್ತು ಸಂಖ್ಯೆಗಳನ್ನು ಹೊರಹಾಕುತ್ತದೆ — ಪ್ರತಿ ಅಂಕೆಗೆ ಒಂದು. ಇದು ಮತದಾನದ ಸ್ಲಿಪ್‌ನಂತೆ: ಹತ್ತು ಅಭ್ಯರ್ಥಿಗಳ ಪೈಕಿ ಒಬ್ಬರ ಮುಂದೆ ಮಾತ್ರ ಮುದ್ರೆ.

ಹಂತ ೪: ನೆಟ್‌ವರ್ಕ್ (ನನ್ನ 6.1 ಪ್ರಯೋಗದ ಅತ್ಯುತ್ತಮ ಮಾದರಿ)



ಚಿತ್ರ ೭.೧: 784 → 512 → 256 → 128 → 64 → 10. ಗುಪ್ತ ಪದರಗಳಲ್ಲಿ ReLU, ಕೊನೆಯ ಪದರದಲ್ಲಿ softmax

ಪದರ	ಪ್ಯಾರಾಮೀಟರ್ ಲೆಕ್ಕ (ಇನ್‌ಪುಟ್ × ನ್ಯೂರಾನ್ + ಬಯಾಸ್)	ಸಂಖ್ಯೆ
Dense 512	784 × 512 + 512	4,01,920
Dense 256	512 × 256 + 256	1,31,328
Dense 128	256 × 128 + 128	32,896
Dense 64	128 × 64 + 64	8,256
Dense 10 (softmax)	64 × 10 + 10	650
ಒಟ್ಟು		**5,75,050**

ಹಂತ ೫: Softmax ಮತ್ತು ಲಾಸ್ — ಕೈಯಿಂದ ಲೆಕ್ಕ

ಕೊನೆಯ ಪದರ ಹತ್ತು “ಹಸಿ ಅಂಕಗಳನ್ನು” (logits) ಕೊಡುತ್ತದೆ. **Softmax** ಅವನ್ನು ಒಟ್ಟು 1 ಆಗುವ ಸಂಭವನೀಯತೆಗಳಾಗಿ ಬದಲಿಸುತ್ತದೆ. **Categorical cross-entropy** = −ln(ನಿಜವಾದ ಅಂಕೆಯ ಸಂಭವನೀಯತೆ).

ಮೂರು ಅಭ್ಯರ್ಥಿಗಳ ಸರಳ ಉದಾಹರಣೆ

logits = [2.0, 1.0, 0.1]. $e^2 = 7.389$, $e^1 = 2.718$, $e^{0.1} = 1.105$; ಮೊತ್ತ = 11.212. → ಸಂಭವನೀಯತೆಗಳು = [**0.659**, 0.242, 0.099] (ಒಟ್ಟು 1). ನಿಜವಾದ ವರ್ಗ ಮೊದಲನೆಯದಾದರೆ: ಲಾಸ್ = $-\ln(0.659)$ = **0.417** (ಕಡಿಮೆ, ಒಳ್ಳೆಯದು). ನಿಜವಾದ ವರ್ಗ ಎರಡನೆಯದಾದರೆ: ಲಾಸ್ = $-\ln(0.242)$ = **1.417** (ಹೆಚ್ಚು, ಶಿಕ್ಷೆ).

10 ಅಂಕೆಗಳ ಉದಾಹರಣೆ (ನಿಜ ಅಂಕೆ = 7)

logits = [1, 2, 8, 1, 0, 1, 1, 6, 1, 0] → softmax: ಅಂಕೆ 2 ಗೆ **0.875**, ಅಂಕೆ 7 ಗೆ **0.118**, ಉಳಿದವು ≈ 0.001–0.002. ನೆಟ್‌ವರ್ಕ್ "2" ಎಂದು ಹೇಳಿತು (ತಪ್ಪು!) ಮತ್ತು ಲಾಸ್ = $-\ln(0.118)$ = **2.13** — ತಪ್ಪಾದ ಮೇಲೆ ಖಚಿತವಾಗಿದ್ದರೆ ಲಾಸ್ ದೊಡ್ಡದು.

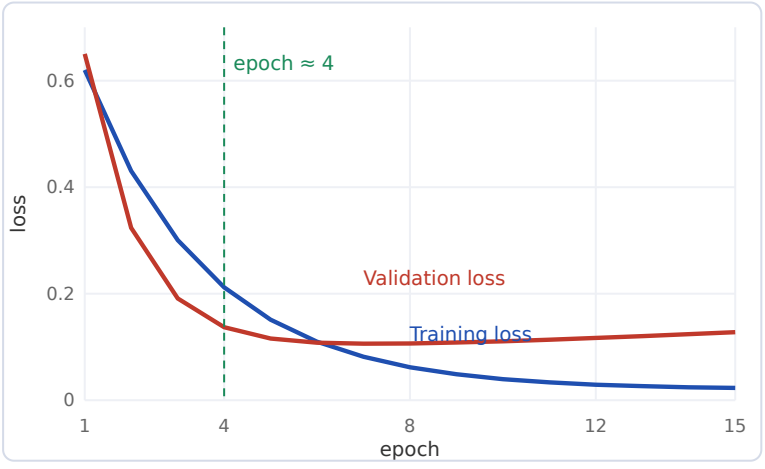
Accuracy vs Loss

Accuracy ಕೇಳುವುದು "ಮೊದಲ ಉತ್ತರ ಸರಿಯೇ?" ಎಂದು ಮಾತ್ರ. Loss ಕೇಳುವುದು "ಎಷ್ಟು ಖಚಿತತೆಯಿಂದ ಹೇಳಿದೆ?" ಎಂದೂ. ಮಾಡೆಲ್ ಅತಿಯಾದ ಆತ್ಮವಿಶ್ವಾಸದಿಂದ ತಪ್ಪು ಹೇಳಿದರೆ accuracy ಹಾಗೇ ಇದ್ದರೂ loss ಹೆಚ್ಚಾಗಬಹುದು. 6.2 ರಲ್ಲಿ ReLU ಗೆ ಹೆಚ್ಚು accuracy ಆದರೆ ಹೆಚ್ಚು loss ಬಂದದ್ದು ಇದೇ ಕಾರಣಕ್ಕೆ.

ಹಂತ ೬: ಡೇಟಾ ವಿಭಜನೆ ಮತ್ತು ಕಲಿಕೆಯ ವಕ್ರರೇಖೆಗಳು

ಪರೀಕ್ಷೆಯ ಹೋಲಿಕೆ

ವಿದ್ಯಾರ್ಥಿ ಕೆಲವು ಹಳೆಯ ಪ್ರಶ್ನೆಪತ್ರಿಕೆಗಳನ್ನು (**training**) ಅಭ್ಯಾಸ ಮಾಡುತ್ತಾನೆ. ಮನೆಯಲ್ಲಿ ಮಾದರಿ ಪರೀಕ್ಷೆ (**validation**) ಬರೆದು ತನ್ನ ಅಧ್ಯಯನ ವಿಧಾನ ಸರಿಪಡಿಸುತ್ತಾನೆ. ಕೊನೆಯಲ್ಲಿ ಅಸಲಿ ಪರೀಕ್ಷೆ (**test**) — ಅದನ್ನು ಮೊದಲೇ ಯಾರೂ ನೋಡಿರಬಾರದು.



ಚಿತ್ರ ೭.೨: Overfitting (ಸ್ಕೀಮ್ಯಾಟಿಕ್ ಚಿತ್ರ). ತರಬೇತಿ loss ಇಳಿಯುತ್ತಲೇ ಇದೆ; validation loss ಸುಮಾರು epoch 4 ರ ನಂತರ ನಿಂತಿದೆ

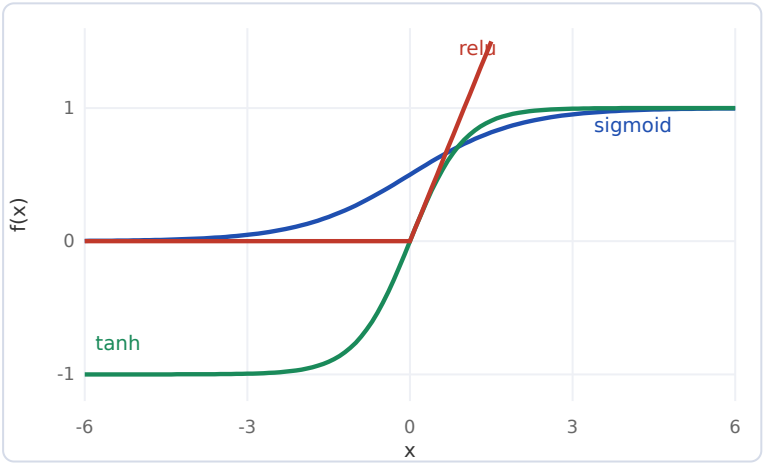
ಲ್ಯಾಬ್‌ನ ನಿಜವಾದ ವಕ್ರರೇಖೆ

ನನ್ನ ಅತ್ಯುತ್ತಮ ಮಾದರಿಯಲ್ಲಿ ತರಬೇತಿ loss ಇಳಿಯುತ್ತಲೇ ಹೋದರೂ validation loss ಸುಮಾರು **epoch 4** ರ ಬಳಿ ಅತ್ಯಂತ ಕಡಿಮೆ ಆಗಿ, ನಂತರ ≈ 0.1 ರಲ್ಲಿ ಸ್ಥಿರವಾಯಿತು. ಇದೇ **overfitting** — ಮಾಡೆಲ್ ಅರ್ಥ ಮಾಡಿಕೊಳ್ಳುವ ಬದಲು ಉರು ಹೊಡೆಯಲು ಶುರು ಮಾಡಿತು. (ಸುಳಿವು: epoch 5 ರ ಸುಮಾರಿಗೇ ನಿಲ್ಲಿಸಿದರೆ — early stopping — ಅದೇ ಗುಣಮಟ್ಟ ಬಹಳ ಬೇಗ ಸಿಗುತ್ತದೆ.)

೮. ಕಾರ್ಯ ೬.೨ — ಆಕ್ಟಿವೇಶನ್ ಫಂಕ್ಷನ್‌ಗಳ ಹೋಲಿಕೆ

ಆಕ್ಟಿವೇಶನ್ ಇಲ್ಲದಿದ್ದರೆ ಏನಾಗುತ್ತದೆ?

ಪ್ರತಿ ಪದರ ಬರೀ ಗುಣಾಕಾರ+ಕೂಡುವಿಕೆ. ಎರಡು ನೇರ ರೇಖೆಗಳನ್ನು ಸೇರಿಸಿದರೂ ಇನ್ನೊಂದು ನೇರ ರೇಖೆಯೇ ಸಿಗುತ್ತದೆ. ಹಾಗಾಗಿ 10 ಪದರ ಇದ್ದರೂ ಇಡೀ ನೆಟ್‌ವರ್ಕ್ ಒಂದೇ ರೇಖೀಯ ಸೂತ್ರಕ್ಕೆ ಕುಸಿಯುತ್ತದೆ. ಆಕ್ಟಿವೇಶನ್ ಆ ಸೂತ್ರಕ್ಕೆ “ಬಾಗು” ಕೊಡುತ್ತದೆ — ಆಗಲೇ ವಕ್ರ ಗಡಿರೇಖೆಗಳನ್ನು ಕಲಿಯಲು ಸಾಧ್ಯ.



ಚಿತ್ರ ೮.೧: ಮೂರು ಆಕ್ಟಿವೇಶನ್ ಕರ್ವ್‌ಗಳು

x	sigmoid	tanh	relu	sigmoid ಗ್ರೇಡಿಯೆಂಟ್
−2	0.119	−0.964	0	0.105
0	0.500	0.000	0	0.250 (ಗರಿಷ್ಠ)
2	0.881	0.964	2	0.105

ಆಕ್ಟಿವೇಶನ್	ಹರವು	ಸ್ವಭಾವ
sigmoid	0 … 1	ಗ್ರೇಡಿಯೆಂಟ್ ≤ 0.25, ಅಂಚುಗಳಲ್ಲಿ ≈ 0. ಆಳವಾದ ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ ಪದರದಿಂದ ಪದರಕ್ಕೆ ಗ್ರೇಡಿಯೆಂಟ್ ಕುಗ್ಗುತ್ತದೆ (**vanishing gradient**) — ಕಲಿಕೆ ನಿಧಾನ
tanh	−1 … 1	0 ಕೇಂದ್ರಿತ, ಗ್ರೇಡಿಯೆಂಟ್ 1 ರವರೆಗೆ. sigmoid ಗಿಂತ ಉತ್ತಮ, ಆದರೂ ಅಂಚುಗಳಲ್ಲಿ saturate ಆಗುತ್ತದೆ
relu	0 … ∞	max(0, x). x > 0 ಆದರೆ ಗ್ರೇಡಿಯೆಂಟ್ ಸರಿಯಾಗಿ 1; ಲೆಕ್ಕ ಬಹಳ ಅಗ್ಗ. ಗುಪ್ತ ಪದರಗಳ ಪ್ರಮಾಣಿತ ಆಯ್ಕೆ. x < 0 ರಲ್ಲಿ ಸಿಲುಕಿದರೆ ನ್ಯೂರಾನ್ “ಸತ್ತು” ಹೋಗಬಹುದು

Vanishing gradient ಲೆಕ್ಕ

sigmoid ನ ಗರಿಷ್ಠ ಗ್ರೇಡಿಯೆಂಟ್ 0.25. 5 ಪದರಗಳ ಮೂಲಕ ಹಿಂದಕ್ಕೆ ಹೋದರೆ ಸಂಕೇತ ಕನಿಷ್ಠ 0.25^5 = **0.00098** ಕ್ಕೆ ಇಳಿಯುತ್ತದೆ; 10 ಪದರಗಳಲ್ಲಿ $0.25^{10} \approx$ **0.00000095**. ಕೊನೆಯ ಪದರದ ದೂರು ಮೊದಲ ಪದರಕ್ಕೆ ತಲುಪುವುದೇ ಇಲ್ಲ — ವಾಟ್ಸ್‌ಆ್ಯಪ್ ಫಾರ್ವರ್ಡ್ ಸರಪಳಿಯಲ್ಲಿ ಮಾಹಿತಿ ಪ್ರತಿ ಹಂತದಲ್ಲೂ ನಾಲ್ಕನೇ ಒಂದು ಭಾಗ ಮಾತ್ರ ಉಳಿದಂತೆ. ReLU ನ ಗ್ರೇಡಿಯೆಂಟ್ 1 ಆದ್ದರಿಂದ ಈ ಸಮಸ್ಯೆ ಕಡಿಮೆ.

ಕೈಬೆರಳ ನಿಯಮ

ಗುಪ್ತ ಪದರಗಳಿಗೆ: relu · **ಹೌದು/ಇಲ್ಲ ಔಟ್‌ಪುಟ್‌ಗೆ:** sigmoid · **10 ವರ್ಗಗಳ ಔಟ್‌ಪುಟ್‌ಗೆ:** softmax.

೯. ಕಾರ್ಯ ೬.೩ — ಆಪ್ಟಿಮೈಜರ್ ಮತ್ತು ಲರ್ನಿಂಗ್ ರೇಟ್

ಆಪ್ಟಿಮೈಜರ್ ಎಂದರೆ ಗ್ರೇಡಿಯೆಂಟ್ ಬಳಸಿ ತೂಕಗಳನ್ನು ಹೇಗೆ ಚಲಿಸಬೇಕು ಎಂದು ನಿರ್ಧರಿಸುವ ನಿಯಮ. ನಿಜವಾದ ಲಾಸ್ ಮೇಲ್ಮೈಗಳು ಒಂದು ದಿಕ್ಕಿನಲ್ಲಿ ಕಡಿದಾಗಿ, ಇನ್ನೊಂದರಲ್ಲಿ ಸಮತಟ್ಟಾಗಿರುತ್ತವೆ (ಉದ್ದನೆಯ ಕಣಿವೆಯಂತೆ). ಮೂರೂ ಆಪ್ಟಿಮೈಜರ್‌ಗಳು ಒಂದೇ ಬಿಂದುವಿನಿಂದ ಶುರುವಾಗುತ್ತವೆ.

ಆಪ್ಟಿಮೈಜರ್	ಕಲ್ಪನೆ	ಸ್ವಭಾವ
SGD	ಹೆಜ್ಜೆ = lr × ಗ್ರೇಡಿಯೆಂಟ್	ಸರಳ; ಆದರೆ lr ಅನ್ನು ಜಾಗ್ರತೆಯಿಂದ ಆರಿಸಬೇಕು. ದೊಡ್ಡದಾದರೆ ಅತ್ತಿತ್ತ ಜಿಗಿದು ಸ್ಫೋಟ; ಚಿಕ್ಕದಾದರೆ ತೆವಳುತ್ತದೆ
RMSprop	ಇತ್ತೀಚಿನ ಗ್ರೇಡಿಯೆಂಟ್‌ಗಳ ಸರಾಸರಿ ಗಾತ್ರದಿಂದ ಹೆಜ್ಜೆಯನ್ನು ಭಾಗಿಸುತ್ತದೆ	ಕಡಿದಾದ ದಿಕ್ಕಿನಲ್ಲಿ ಚಿಕ್ಕ ಹೆಜ್ಜೆ, ಸಮತಟ್ಟಾದ ದಿಕ್ಕಿನಲ್ಲಿ ದೊಡ್ಡ ಹೆಜ್ಜೆ
Adam	Momentum (ಸರಾಸರಿ ದಿಕ್ಕು) + RMSprop ಅಳತೆ	ಸಾಮಾನ್ಯವಾಗಿ ಅತ್ಯಂತ ಸುರಕ್ಷಿತ ಆರಂಭಿಕ ಆಯ್ಕೆ; Keras ನಲ್ಲಿ ಡೀಫಾಲ್ಟ್ lr = 0.001

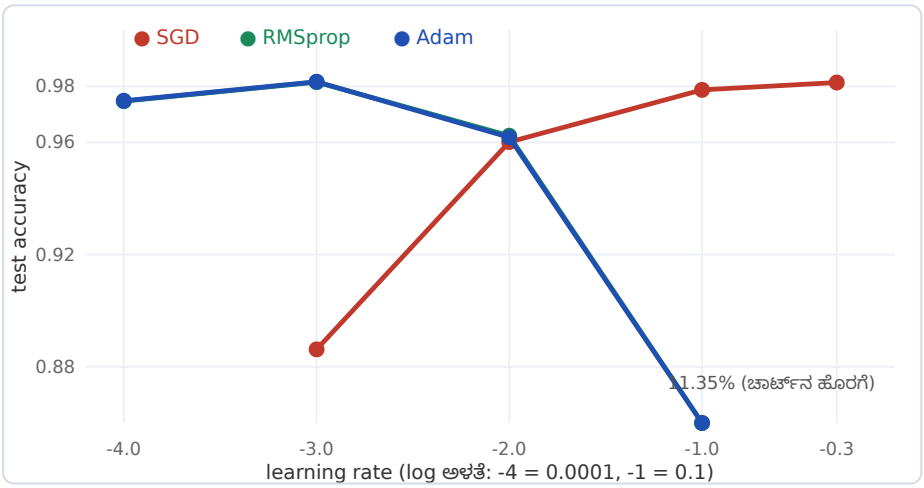
ಸೈಕಲ್ ಸವಾರಿಯ ಹೋಲಿಕೆ

SGD = ಪ್ರತಿ ಕ್ಷಣ ಕಾಲಿನ ಕೆಳಗಿನ ಇಳಿಜಾರನ್ನು ಮಾತ್ರ ನೋಡಿ ಹೆಜ್ಜೆ ಇಡುವವನು. Momentum = ಸೈಕಲ್ ಇಳಿಜಾರಿನಲ್ಲಿ ವೇಗ ಹೆಚ್ಚಿಸಿಕೊಳ್ಳುವಂತೆ, ಹಿಂದಿನ ದಿಕ್ಕನ್ನು ನೆನಪಿಟ್ಟು ಮುಂದುವರಿಯುವುದು. RMSprop = ಕಡಿದಾದ ರಸ್ತೆಯಲ್ಲಿ ಬ್ರೇಕ್ ಹಿಡಿದು, ಸಮತಟ್ಟಾದ ರಸ್ತೆಯಲ್ಲಿ ವೇಗ ಹೆಚ್ಚಿಸುವುದು. Adam = ಇವೆರಡನ್ನೂ ಒಟ್ಟಿಗೆ ಮಾಡುವ ಅನುಭವಿ ಸವಾರ.

Adam ನ ಮೊದಲ ಹೆಜ್ಜೆ — ಲೆಕ್ಕ

loss = w^2, w = 4 → ಗ್ರೇಡಿಯೆಂಟ್ g = 8. m = 0.1×8 = 0.8, v = 0.001×64 = 0.064; bias-correction ನಂತರ $\hat{m}$ = 8, $\hat{v}$ = 64. ಹೆಜ್ಜೆ = lr × $\hat{m}$ ÷ (√$\hat{v}$ + ε) = 0.001 × 8 ÷ 8 = **0.001**. ಗಮನಿಸಿ: ಗ್ರೇಡಿಯೆಂಟ್ 8 ಆಗಿರಲಿ, 800 ಆಗಿರಲಿ, Adam ನ ಹೆಜ್ಜೆ ≈ lr — ಅದಕ್ಕೇ ಅದು ತುಂಬಾ ಸ್ಥಿರ. (SGD ಹೆಜ್ಜೆ lr × 8 — ಗ್ರೇಡಿಯೆಂಟ್ ಜೊತೆಗೇ ಬೆಳೆಯುತ್ತದೆ.)

ನನ್ನ ಫಲಿತಾಂಶಗಳು: ಆಪ್ಟಿಮೈಜರ್ × ಲರ್ನಿಂಗ್ ರೇಟ್ (ಟೆಸ್ಟ್ accuracy)



ಚಿತ್ರ ೯.೧: ಪ್ರತಿ ಆಪ್ಟಿಮೈಜರ್‌ನ accuracy ಲರ್ನಿಂಗ್ ರೇಟ್ ಬದಲಾದಂತೆ (ಮೂಲ ಗ್ರೈಡ್‌ನ ನಿಜ ಸಂಖ್ಯೆಗಳು). lr 0.1 ನಲ್ಲಿ RMSprop/Adam = 11.35% (ಚಾರ್ಟ್‌ನ ಕೆಳಗೆ)

ಆಪ್ಟಿಮೈಜರ್	lr → accuracy
SGD	0.001 → 88.62% · 0.01 → 96.01% · 0.1 → 97.87% · 0.5 → **98.13%**
RMSprop	0.0001 → 97.47% · 0.001 → **98.15%** · 0.01 → 96.24% · 0.1 → **11.35%**
Adam	0.0001 → 97.48% · **0.001 → 98.16%** · 0.01 → 96.18% · 0.1 → **11.35%**

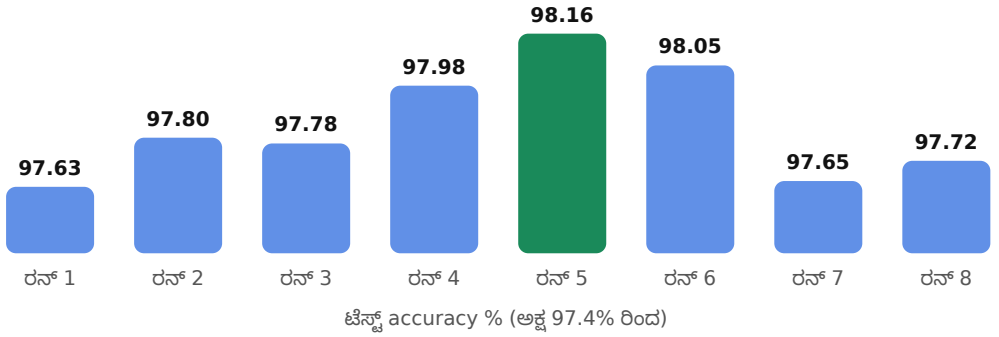
ಓದುವ ವಿಧಾನ

SGD ಗೆ ದೊಡ್ಡ ದರ (0.1–0.5) ಬೇಕು; RMSprop/Adam ಗೆ ಚಿಕ್ಕ ದರ (0.0001–0.001) ಇಷ್ಟ, ಮತ್ತು 0.1 ರಲ್ಲಿ ಅವು ಯಾದೃಚ್ಛಿಕ ಊಹೆಯ ಮಟ್ಟಕ್ಕೆ (11.35% ≈ ಯಾವಾಗಲೂ ಒಂದೇ ಅಂಕೆ ಹೇಳುವುದು) ಕುಸಿದವು. **ಅತ್ಯುತ್ತಮ: Adam, lr 0.001 → 98.16%.** ಒಟ್ಟಿನಲ್ಲಿ **ಆಪ್ಟಿಮೈಜರ್‌ಗಿಂತ ಲರ್ನಿಂಗ್ ರೇಟ್ ಹೆಚ್ಚು ಮುಖ್ಯ** ಎಂದು ಕಂಡುಬಂದಿತು.

೧೦. ನನ್ನ ಪ್ರಯೋಗ ಫಲಿತಾಂಶಗಳು ಒಂದೇ ನೋಟದಲ್ಲಿ

ನೋಟ್‌ಬುಕ್‌ನ ನಿಜವಾದ ಸಂಖ್ಯೆಗಳು (MNIST ಟೆಸ್ಟ್ ಸೆಟ್, 10,000 ಚಿತ್ರಗಳು). ಹಸಿರು ಸಾಲು = ಅತ್ಯುತ್ತಮ.

೬.೧ ಪದರಗಳು, ನ್ಯೂರಾನ್‌ಗಳು, ಬ್ಯಾಚ್ ಗಾತ್ರ, validation split



ಚಿತ್ರ ೧೦.೧: ಎಂಟು ರನ್‌ಗಳ ಟೆಸ್ಟ್ accuracy (ಅಕ್ಷ 97.4% ರಿಂದ ಶುರು — ಸಣ್ಣ ವ್ಯತ್ಯಾಸ ಕಾಣಿಸಲು)

Run	ನ್ಯೂರಾನ್‌ಗಳು	Epochs	Batch	Val split	Train acc	Test acc	Train loss	Test loss
1	128	15	128	0.2	0.9938	0.9763	0.0272	0.0842
2	512	15	128	0.2	0.9938	0.9780	0.0246	0.0914
3	256-128	15	128	0.2	0.9934	0.9778	0.0300	0.1044
4	256-128-64	15	128	0.2	0.9923	0.9798	0.0326	0.0974
5	**512-256-128-64**	15	128	0.2	0.9937	**0.9816**	0.0296	**0.0845**
6	512-256-128-64	15	64	0.2	0.9933	0.9805	0.0317	0.1025
7	512-256-128-64	15	128	0.1	0.9900	0.9765	0.0371	0.1097
8	512-256-128-64	15	128	0.3	0.9890	0.9772	0.0456	0.1008

ತೀರ್ಮಾನ: ಅತ್ಯುತ್ತಮ = ರನ್ 5 (ನಾಲ್ಕು ಗುಪ್ತ ಪದರ 512-256-128-64), **98.16%** ಟೆಸ್ಟ್ accuracy, loss 0.0845. Batch 64 ಮತ್ತು validation 10%/30% ಸ್ವಲ್ಪ ಕೆಟ್ಟವು. ಪರೀಕ್ಷಾ ಅಂಕೆಯನ್ನು ಎಲ್ಲಾ 8 ಮಾದರಿಗಳೂ ಸರಿಯಾಗಿ ಗುರುತಿಸಿದವು.

೬.೨ ಆಕ್ಟಿವೇಶನ್ (ಪ್ರತಿಯೊಂದಕ್ಕೆ 3 ಬಾರಿ ಪುನರಾವರ್ತನೆ)

ಆಕ್ಟಿವೇಶನ್	ರನ್ ೧	ರನ್ ೨	ರನ್ ೩	ಸರಾಸರಿ accuracy
sigmoid	97.31	97.26	97.60	97.39%
tanh	97.44	97.61	97.25	97.43%
relu	97.64	97.71	97.40	**97.58%**

ವ್ಯತ್ಯಾಸ ಚಿಕ್ಕದು (≈ ಒಂದೇ ಆಕ್ಟಿವೇಶನ್‌ನ ರನ್‌ಗಳ ನಡುವಿನ ಏರುಪೇರಿನಷ್ಟೇ). relu ಗೆ ಅತ್ಯುತ್ತಮ accuracy (97.58%), sigmoid ಗೆ ಅತಿ ಕಡಿಮೆ loss (0.093).

> **ಸೂಚಿತ ಅತ್ಯುತ್ತಮ ಮಾದರಿ**
>
> Dense 512-256-128-64, relu, softmax ಔಟ್‌ಪುಟ್, Adam lr 0.001, batch 128, 15 epochs, validation 20% → **98.16%** ಟೆಸ್ಟ್ accuracy. ಸುಮಾರು 5ನೇ epoch ನಲ್ಲಿ early stopping ಮಾಡಿದರೆ ಅದೇ ಗುಣಮಟ್ಟ ಬಹಳ ವೇಗವಾಗಿ ಸಿಗುತ್ತದೆ.

೧೧. ಲ್ಯಾಬ್‌ಗೆ ಬೇಕಾದ ಕೋಡ್ (Python)

ಇವು ಲ್ಯಾಬ್‌ನ ಪ್ರತಿ ಹಂತದ ಚಿಕ್ಕ, ಅರ್ಥವಾಗುವ ಕೋಡ್‌ಗಳು. NumPy/pandas ಭಾಗಗಳನ್ನು ನಾನು ಓಡಿಸಿ ಫಲಿತಾಂಶ ಪರಿಶೀಲಿಸಿದ್ದೇನೆ. Keras ಕೋಡ್ ಈ ಪರಿಸರದಲ್ಲಿ ಓಡಿಸಲಾಗಲಿಲ್ಲ (TensorFlow ಇಲ್ಲ) — ಆದ್ದರಿಂದ ಅದನ್ನು ಓಡಿಸಿ ನೋಡಿ.

೧. ಕಾಣೆಯಾದ ಮೌಲ್ಯ ತುಂಬುವುದು (ಗುಂಪಿನ ಪ್ರಕಾರ)

```
import pandas as pd
df = pd.DataFrame({"gender":[0,1,0,1,0,0,0,0,0],
                   "height":[165,183,171,None,160,169,167,163,None]})
# ಪ್ರತಿ gender ಗುಂಪಿನ ಸರಾಸರಿಯಿಂದ ತುಂಬಿ
df["height"] = df["height"].fillna(df.groupby("gender")["height"].transform("mean"))
print(df)
```

೨. Z-score, Chauvenet ಮತ್ತು IQR

```
import numpy as np
from math import erfc, sqrt
a = np.array([12,13,14,12,15,13,14,16,13,12,14,15,88], float)

# Z-score
z = (a - a.mean()) / a.std(ddof=1)
print("z>3:", a[np.abs(z) > 3])

# Chauvenet: n * P(|Z| >= z) < 0.5
n = len(a)
p = np.array([erfc(abs(v)/sqrt(2)) for v in z])
print("chauvenet:", a[n*p < 0.5])

# IQR (quartile) fences
q1, q3 = np.percentile(a, [25, 75]); iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
print("fences:", lo, hi, "outliers:", a[(a < lo) | (a > hi)])
```

೩. ಸ್ಕೇಲಿಂಗ್ (ಟ್ರೈನಿಂಗ್ ಅಂಕಿಗಳನ್ನೇ ಬಳಸಿ)

```
h = np.array([150,160,170,180,190], float)
lo, hi = h.min(), h.max()          # ಟ್ರೈನಿಂಗ್ ಡೇಟಾದಿಂದ
print((h - lo) / (hi - lo))        # min-max
print((h - h.mean()) / h.std())    # standardize
```

೪. ಸಣ್ಣ ನೆಟ್‌ವರ್ಕ್‌ನ ಒಂದು ಹೆಜ್ಜೆ (ಭಾಗ ೫ ರ ಲೆಕ್ಕ)

```
from math import exp
sig = lambda x: 1/(1+exp(-x))
x1, x2, truth, lr = 0.5, 0.4, 1, 0.5
w1,w2,b1 = 0.5,-0.3,0.1;  w3,w4,b2 = -0.2,0.4,0.0;  w5,w6,b3 = 0.6,-0.5,0.2

# ಮುಂದಕ್ಕೆ
h1 = sig(w1*x1 + w2*x2 + b1); h2 = sig(w3*x1 + w4*x2 + b2)
out = sig(w5*h1 + w6*h2 + b3)
# ಹಿಂದಕ್ಕೆ
d_out = (out - truth) * out * (1 - out)
d_h1 = d_out * w5 * h1 * (1 - h1); d_h2 = d_out * w6 * h2 * (1 - h2)
# ತೂಕ ಬದಲಿಸಿ
w5 -= lr*d_out*h1; w6 -= lr*d_out*h2; b3 -= lr*d_out
w1 -= lr*d_h1*x1;  w2 -= lr*d_h1*x2;  b1 -= lr*d_h1
w3 -= lr*d_h2*x1;  w4 -= lr*d_h2*x2;  b2 -= lr*d_h2
h1 = sig(w1*x1 + w2*x2 + b1); h2 = sig(w3*x1 + w4*x2 + b2)
print(out, "->", sig(w5*h1 + w6*h2 + b3))   # 0.5688 -> 0.5898
```

೫. MNIST ಮಾದರಿ (Keras)

```
from tensorflow import keras
from tensorflow.keras import layers

(x_tr, y_tr), (x_te, y_te) = keras.datasets.mnist.load_data()
x_tr = x_tr.reshape(-1, 784) / 255.0          # flatten + 0-1
x_te = x_te.reshape(-1, 784) / 255.0
y_tr = keras.utils.to_categorical(y_tr, 10)   # one-hot
y_te = keras.utils.to_categorical(y_te, 10)

model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(512, activation="relu"),
    layers.Dense(256, activation="relu"),
    layers.Dense(128, activation="relu"),
    layers.Dense(64,  activation="relu"),
    layers.Dense(10,  activation="softmax"),
])
model.compile(optimizer=keras.optimizers.Adam(0.001),
              loss="categorical_crossentropy", metrics=["accuracy"])
hist = model.fit(x_tr, y_tr, epochs=15, batch_size=128,
                 validation_split=0.2)
print(model.evaluate(x_te, y_te))              # ~[0.08, 0.98]
model.summary()                                # 575,050 parameters
```

೧೨. ಏನೆಲ್ಲಾ ಕಲಿಯಬೇಕು — ಪರಿಶೀಲನಾ ಪಟ್ಟಿ

ಒಂದೊಂದನ್ನು ಕರಗತ ಮಾಡಿಕೊಂಡಂತೆ ☐ ಅನ್ನು ✔ ಮಾಡಿಕೊಳ್ಳಿ.

ವಿಭಾಗ	ಕಲಿಯಬೇಕಾದದ್ದು	ಯಾಕೆ?
ಅಡಿಪಾಯ	☐ Python, numpy ಅರೇ, pandas DataFrame	ಆರೂ ಕಾರ್ಯಗಳು ಕೋಡ್; indexing, boolean mask, groupby ಯಾವಾಗಲೂ ಬಳಕೆ
	☐ matplotlib ಮೂಲಗಳು	z-score, loss ಮತ್ತು learning curve ಗಳ ಚಿತ್ರಗಳು
	☐ Mean, median, std, percentile	ಕಾರ್ಯ ೧–೪ ರ ಭಾಷೆ
	☐ ಸಾಮಾನ್ಯ (ಬೆಲ್) ವಿತರಣೆ ಮತ್ತು ಸಂಭವನೀಯತೆ	“3σ” ಏಕೆ ಅಪರೂಪ; Chauvenet ಇದನ್ನೇ ಬಳಸುತ್ತದೆ
	☐ ವೆಕ್ಟರ್, dot product, matrix ಗುಣಾಕಾರ	Dense ಪದರ = x · W + b

ಡೇಟಾ ಸಿದ್ಧತೆ	☐ ಕಾಣೆಯಾದ ಮೌಲ್ಯ ಮತ್ತು imputation	ಕಾರ್ಯ ೧: ಗುಂಪಿನ ಪ್ರಕಾರ mean vs median
	☐ Outliers: Z-score, Chauvenet, IQR	ಕಾರ್ಯ ೨–೪ ಮತ್ತು ಅವು ಯಾವಾಗ ಸೋಲುತ್ತವೆ (masking)
	☐ Normalization: min-max, standardization	ಏಕೆ ಸ್ಕೇಲ್ ಮಾಡಬೇಕು; ಟ್ರೈನ್ ಡೇಟಾದಿಂದಲೇ fit
	☐ Train / validation / test ವಿಭಜನೆ	ಪ್ರಾಮಾಣಿಕ ಮೌಲ್ಯಮಾಪನ; Keras ನ validation_split
	☐ One-hot encoding	categorical_crossentropy ಗೆ ಅಗತ್ಯ
ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕೋರ್	☐ ನ್ಯೂರಾನ್: ತೂಕ, ಬಯಾಸ್, ಆಕ್ಟಿವೇಶನ್	ಕಾರ್ಯ ೫ ರ ಚಿತ್ರ
	☐ Sigmoid, tanh, relu, softmax	ಕಾರ್ಯ ೬.೨; ಔಟ್‌ಪುಟ್‌ನಲ್ಲಿ softmax ಏಕೆ
	☐ Forward pass	ಊಹೆ ಹೇಗೆ ಹುಟ್ಟುತ್ತದೆ
	☐ Loss: MSE, binary/categorical cross-entropy	ನೆಟ್‌ವರ್ಕ್ ಕನಿಷ್ಠಗೊಳಿಸಲು ಯತ್ನಿಸುವುದು
	☐ Gradient descent ಮತ್ತು backpropagation	ತೂಕ ಹೇಗೆ ಬದಲಾಗುತ್ತದೆ (chain rule)
	☐ Learning rate, epochs, batch size	ಕಾರ್ಯ ೬.೧–೬.೩ ರ ಮುಖ್ಯ ನಾಬ್‌ಗಳು
Keras ಮತ್ತು MNIST	☐ Sequential, Dense, Input	ಕೆಲವೇ ಸಾಲುಗಳಲ್ಲಿ ಮಾದರಿ ಕಟ್ಟುವುದು
	☐ compile / fit / evaluate / predict	ಸಂಪೂರ್ಣ Keras ಕೆಲಸದ ಹರಿವು
	☐ ಆಪ್ಟಿಮೈಜರ್: SGD, RMSprop, Adam	ಕಾರ್ಯ ೬.೩
	☐ Overfitting ಮತ್ತು learning curves	train vs validation ವಕ್ರರೇಖೆ ಓದುವುದು
	☐ ನ್ಯಾಯಯುತ ಪ್ರಯೋಗ ವಿನ್ಯಾಸ	ಒಮ್ಮೆಗೆ ಒಂದೇ hyperparameter ಬದಲಿಸಿ; ಏರುಪೇರು ನೋಡಲು ಪುನರಾವರ್ತಿಸಿ

ವರದಿ ಮತ್ತು ವೈವಾ	☐ ಫಲಿತಾಂಶ ಕೋಷ್ಟಕ ಮತ್ತು ತೀರ್ಮಾನ ಬರೆಯುವುದು	ವರದಿಗೆ: epochs, accuracy, loss, batch, split
	☐ ನಿಮ್ಮ ಕೋಡ್ ಅನ್ನು ಸಾಲು-ಸಾಲಾಗಿ ವಿವರಿಸುವುದು	ವೈವಾ ಸಮಯದಲ್ಲಿ ಕೋಡ್ ತೆರೆದಿರಬೇಕು
	☐ ನಿಮ್ಮ ಪ್ರಯೋಗದ ಮಿತಿಗಳನ್ನು ಒಪ್ಪಿಕೊಳ್ಳುವುದು	test ಸೆಟ್‌ನಿಂದ ಮಾದರಿ ಆರಿಸಿದ್ದು; ಒಂದೇ ಪರೀಕ್ಷಾ ಅಂಕೆ; ಕೇವಲ 3 ಪುನರಾವರ್ತನೆ

೧೩. ವೈವಾ (Defense) ಪ್ರಶ್ನೋತ್ತರಗಳು

ಇವನ್ನು ಬಾಯಿಯಲ್ಲಿ ಹೇಳಲು ಅಭ್ಯಾಸ ಮಾಡಿ. ನೆನಪಿಡಿ: ವೈವಾದಲ್ಲಿ ಕೋಡ್ ತೆರೆದಿರಬೇಕು ಮತ್ತು ಪ್ರತಿ ಸೆಲ್ ಅನ್ನು ವಿವರಿಸಲು ಸಾಧ್ಯವಾಗಬೇಕು.

ಪ್ರಶ್ನೆ	ಉತ್ತರ (ಈ ಲ್ಯಾಬ್‌ನ ಸಂಖ್ಯೆಗಳೊಂದಿಗೆ)
೧. ಎತ್ತರ ತುಂಬುವಾಗ ಲಿಂಗವನ್ನು ಏಕೆ ಬಳಸಿದಿರಿ?	ಲಿಂಗದ ಪ್ರಕಾರ ಎತ್ತರ ಬಹಳ ಬದಲಾಗುತ್ತದೆ (ಮಹಿಳೆಯರು ≈ 165.8 cm, ಪುರುಷರು ≈ 182.7 cm). ಎಲ್ಲರ ಸರಾಸರಿ (~175) ಬಳಸಿದರೆ ಮಹಿಳೆಗೆ ಸುಮಾರು 9 cm ತಪ್ಪಾಗುತ್ತದೆ. ಅದಕ್ಕೇ ಅದೇ ಗುಂಪಿನ ಮೌಲ್ಯಗಳನ್ನೇ ಬಳಸಿದೆ.
೨. ಇಂಪ್ಯೂಟೇಶನ್‌ಗೆ mean ಅಥವಾ median?	ಎರಡೂ ಬಹುತೇಕ ಒಂದೇ ಫಲಿತಾಂಶ ಕೊಟ್ಟವು (165.83 vs 166 cm). ವಿಪರೀತ ಮೌಲ್ಯಗಳಿದ್ದರೆ median ಸುರಕ್ಷಿತ, ಏಕೆಂದರೆ ಒಂದು ತೀವ್ರ ಮೌಲ್ಯ ಅದನ್ನು ಕದಲಿಸುವುದಿಲ್ಲ.
೩. ಮೂರು ಅಸಹಜ-ಮೌಲ್ಯ ವಿಧಾನಗಳ ವ್ಯತ್ಯಾಸ?	Z-score ಮತ್ತು Chauvenet mean ಹಾಗೂ std ಬಳಸುತ್ತವೆ (ಸಾಮಾನ್ಯ ವಿತರಣೆ ಊಹೆ); ಅಸಹಜ ಮೌಲ್ಯವೇ σ ಅನ್ನು ಉಬ್ಬಿಸುತ್ತದೆ. Chauvenet ಮಾದರಿ ಗಾತ್ರ n ಅನ್ನೂ ಪರಿಗಣಿಸುತ್ತದೆ. IQR quartile ಬಳಸುತ್ತದೆ, ಅಸಹಜ ಮೌಲ್ಯ ಅದನ್ನು ವಿರೂಪಗೊಳಿಸಲಾರದು. ಈ ಲ್ಯಾಬ್‌ನಲ್ಲಿ ಮೂರೂ 88 ಅನ್ನು ಮಾತ್ರ ಹಿಡಿದವು.
೪. Z-score ಅಸಹಜ ಮೌಲ್ಯವನ್ನು ಏಕೆ ತಪ್ಪಿಸಬಹುದು?	Masking: ದೊಡ್ಡ ಅಸಹಜ ಮೌಲ್ಯ σ ಅನ್ನು ಹೆಚ್ಚಿಸಿ ತನ್ನ ಸ್ವಂತ z-score ಅನ್ನು ಕುಗ್ಗಿಸುತ್ತದೆ. 88 ಇರುವಾಗ σ = 14.04, ಇಲ್ಲದಿದ್ದರೆ ಕೇವಲ 2.4. ಮೌಲ್ಯ 30–40 ಇದ್ದರೆ Z-score ವಿಫಲವಾಗಬಹುದು, ಆದರೆ IQR ಕೆಲಸ ಮಾಡುತ್ತದೆ.
೫. ಇನ್‌ಪುಟ್‌ಗಳನ್ನು [0,1] ಗೆ ಏಕೆ normalize ಮಾಡಿದಿರಿ?	ಎತ್ತರ ಮತ್ತು ತೂಕಕ್ಕೆ ಬೇರೆ ಬೇರೆ ಘಟಕ ಮತ್ತು ವ್ಯಾಪ್ತಿ. ಸ್ಕೇಲ್ ಮಾಡಿದರೆ ತೂಕ ಮತ್ತು ಗ್ರೇಡಿಯೆಂಟ್‌ಗಳು ಒಂದೇ ಗಾತ್ರದಲ್ಲಿರುತ್ತವೆ; ತರಬೇತಿ ಸ್ಥಿರ ಮತ್ತು ವೇಗ. min ಮತ್ತು max ಟ್ರೈನಿಂಗ್ ಡೇಟಾದಿಂದ ತೆಗೆದು ಹೊಸ ಡೇಟಾಕ್ಕೂ ಅದನ್ನೇ ಬಳಸುತ್ತೇನೆ.
೬. ಬ್ಯಾಕ್‌ಪ್ರೊಪಗೇಶನ್ ಅನ್ನು ಒಂದು ನಿಮಿಷದಲ್ಲಿ ವಿವರಿಸಿ.	Forward pass ಔಟ್‌ಪುಟ್ ಲೆಕ್ಕ ಹಾಕುತ್ತದೆ; ಲಾಸ್ ಅದನ್ನು ಸತ್ಯದ ಜೊತೆ ಹೋಲಿಸುತ್ತದೆ; chain rule ಬಳಸಿ ಔಟ್‌ಪುಟ್‌ನಿಂದ ಹಿಂದಕ್ಕೆ ಪ್ರತಿ ತೂಕದ ಗ್ರೇಡಿಯೆಂಟ್ ಸಿಗುತ್ತದೆ; ಪ್ರತಿ ತೂಕವನ್ನು ಲಾಸ್ ಕಡಿಮೆ ಮಾಡುವ ದಿಕ್ಕಿನಲ್ಲಿ (learning rate ಪ್ರಮಾಣದಲ್ಲಿ) ಸ್ವಲ್ಪ ಸರಿಸುತ್ತೇವೆ.

೭. ಬಯಾಸ್ ಏನು ಮಾಡುತ್ತದೆ?	ಆಕ್ಟಿವೇಶನ್‌ನ ಮಿತಿಯನ್ನು ಸರಿಸುತ್ತದೆ — ನೇರ ರೇಖೆಯ intercept ನಂತೆ. ಕಾರ್ಯ ೫ ರಲ್ಲಿ ಇದು 1 ಎಂದು ಸ್ಥಿರವಾದ ಹೆಚ್ಚುವರಿ ಇನ್‌ಪುಟ್ ಹಾಗೂ ತನ್ನದೇ ತೂಕ, ಯಾದೃಚ್ಛಿಕವಾಗಿ ಆರಂಭಿಸಲಾಗಿದೆ.
೮. ಕಾರ್ಯ ೫ ರಲ್ಲಿ ಬೇರೆ lr ಮತ್ತು epochs ನಿಂದ ಏನಾಯಿತು?	lr 0.01 / 100 epochs ಬಹುತೇಕ ಕಲಿಯಲಿಲ್ಲ (accuracy 53%, ತಪ್ಪು ಉತ್ತರ). lr 0.1 ಮತ್ತು 1000 epochs ನಿಂದ loss ≈ 0.005, accuracy 100%, Name 5 = ಮಹಿಳೆ (ಔಟ್‌ಪುಟ್ ≈ 0.02). ಹೆಚ್ಚಿನ lr ಅಥವಾ epochs ಲಾಸ್ ಅನ್ನು ಇನ್ನೂ ಕಡಿಮೆ ಮಾಡುತ್ತವೆ.
೯. MNIST ಗೆ softmax ಮತ್ತು categorical cross-entropy ಏಕೆ?	10 ಪರಸ್ಪರ ಹೊರಗಿರುವ ವರ್ಗಗಳು: softmax ಒಟ್ಟು 1 ಆಗುವ ಸಂಭವನೀಯತೆ ಕೊಡುತ್ತದೆ; cross-entropy = −ln(ನಿಜವಾದ ವರ್ಗದ ಸಂಭವನೀಯತೆ), ಆತ್ಮವಿಶ್ವಾಸದ ತಪ್ಪುಗಳಿಗೆ ಬಲವಾದ ಶಿಕ್ಷೆ.
೧೦. 6.1 ರಲ್ಲಿ ಅತ್ಯುತ್ತಮ ಮಾದರಿ ಯಾವುದು ಮತ್ತು ಹೇಗೆ ಗೊತ್ತು?	512-256-128-64, relu, Adam: 98.16% ಟೆಸ್ಟ್ accuracy, loss 0.0845 — 8 ರನ್‌ಗಳಲ್ಲಿ ಅತ್ಯಧಿಕ accuracy. ರನ್‌ಗಳ ನಡುವಿನ ವ್ಯತ್ಯಾಸ ಚಿಕ್ಕದು (≈ 0.5%), ಹಾಗಾಗಿ “ಸ್ವಲ್ಪ ಉತ್ತಮ” ಎನ್ನುತ್ತೇನೆ, “ತುಂಬಾ ಉತ್ತಮ” ಅಲ್ಲ.
೧೧. ಕಲಿಕೆಯ ವಕ್ರರೇಖೆ ಏನು ತೋರಿಸುತ್ತದೆ?	ಟ್ರೈನಿಂಗ್ loss ಇಳಿಯುತ್ತಲೇ ಇದೆ; validation loss ಸುಮಾರು epoch 4 ರ ಹತ್ತಿರ ಅತಿ ಕಡಿಮೆ, ನಂತರ ≈ 0.1 ರ ಬಳಿ ಸ್ಥಿರ. ಇದು overfitting — ಮಾದರಿ ಟ್ರೈನಿಂಗ್ ಸೆಟ್ ಅನ್ನು ಉರು ಹೊಡೆಯಲು ಶುರು ಮಾಡಿದೆ.
೧೨. ಯಾವ ಆಕ್ಟಿವೇಶನ್ ಉತ್ತಮ?	relu ಸರಾಸರಿ accuracy ಅತ್ಯುತ್ತಮ (97.58%) vs tanh 97.43%, sigmoid 97.39%. ಆದರೆ ವ್ಯತ್ಯಾಸ ಪುನರಾವರ್ತನೆಗಳ ಏರುಪೇರಿನಷ್ಟೇ. sigmoid ಗೆ ಅತಿ ಕಡಿಮೆ loss (0.093). ಹಾಗಾಗಿ ತೀರ್ಮಾನ ಜಾಗ್ರತೆಯಿಂದ ಹೇಳಬೇಕು.
೧೩. Adam ಮತ್ತು RMSprop lr 0.1 ನಲ್ಲಿ ಏಕೆ ವಿಫಲ?	ಹೆಜ್ಜೆ ಬಹಳ ದೊಡ್ಡದು; ತೂಕಗಳು ಕನಿಷ್ಠ ಬಿಂದುವನ್ನು ದಾಟಿ ಜಿಗಿದು ತರಬೇತಿ ಅಸ್ಥಿರವಾಗುತ್ತದೆ; ಕೊನೆಗೆ ಊಹೆಯ ಮಟ್ಟ (11.35%) ಗೆ ಬೀಳುತ್ತದೆ. SGD ಯ ಹೆಜ್ಜೆ ಚಿಕ್ಕ, ಅದಕ್ಕೆ ದೊಡ್ಡ ದರ (0.1–0.5) ಬೇಕು.
೧೪. ನಿಮ್ಮ ಪ್ರಯೋಗದ ದೌರ್ಬಲ್ಯಗಳೇನು?	ಅತ್ಯುತ್ತಮ ಮಾದರಿಯನ್ನು test ಸೆಟ್‌ನಲ್ಲೇ ಆರಿಸಿದೆ (ಅಸೈನ್‌ಮೆಂಟ್ ಹಾಗೆ ಕೇಳುತ್ತದೆ); ಆಕ್ಟಿವೇಶನ್‌ಗೆ ಕೇವಲ 3 ಪುನರಾವರ್ತನೆ; ಒಂದೇ ಯಾದೃಚ್ಛಿಕ ವಿಭಜನೆ; ಸ್ಥಿರ 15 epochs; ಒಂದೇ ಪರೀಕ್ಷಾ ಅಂಕೆ. ನಿಜವಾದ ಯೋಜನೆಯಲ್ಲಿ ಆಯ್ಕೆಗೆ ಪ್ರತ್ಯೇಕ validation ಸೆಟ್, test ಸೆಟ್ ಒಮ್ಮೆ ಮಾತ್ರ.

ಪ್ರತಿ ಕಾರ್ಯಕ್ಕೆ ನೋಟ್‌ಬುಕ್‌ನಲ್ಲಿ ಎಲ್ಲಿ ನೋಡಬೇಕು?

ಕಾರ್ಯ	ಪರಿಕಲ್ಪನೆ	ನೋಟ್‌ಬುಕ್‌ನಲ್ಲಿ	ಮುಖ್ಯ ಔಟ್‌ಪುಟ್
1	imputation, groupby	“Task 1” ಸೆಲ್‌ಗಳು	165.83 / 166 cm
2	Z-score	z = (A−mean)/std	88, z = 5.22
3	Chauvenet	erfc, n·P < 0.5	88, n·P = 5.4e-6
4	IQR	np.percentile	ಮಿತಿ 4.625 / 19.625
5	scratch ನಿಂದ backprop	class SmallNet	Name 5 → female
6.1	ಆಳ, ಅಗಲ, batch, validation	run_model(...)	98.16%, 512-256-128-64
6.2	ಆಕ್ಟಿವೇಶನ್‌ಗಳು	act × 3 ಪುನರಾವರ್ತನೆ ಲೂಪ್	relu 97.58%
6.3	ಆಪ್ಟಿಮೈಜರ್‌ಗಳು	sgd / rmsprop / adam ಗ್ರಿಡ್	Adam 0.001, 98.16%

ಕೊನೆಯ ಮಾತು

ಈ ಲ್ಯಾಬ್‌ನ ಮುಖ್ಯ ಪಾಠ ಒಂದೇ: **ಮೊದಲು ಡೇಟಾವನ್ನು ನಂಬಲರ್ಹವಾಗಿಸಿ, ನಂತರ ಮಾದರಿ ಕಟ್ಟಿ, ಕೊನೆಗೆ ಒಮ್ಮೆಗೆ ಒಂದೇ ಬದಲಾವಣೆ ಮಾಡಿ ಹೋಲಿಸಿ — ಮತ್ತು ಹಿಂದೆಂದೂ ನೋಡದ ಡೇಟಾದ ಮೇಲೆ ಅಳೆಯಿರಿ.** ಇದನ್ನು ಮಾಡಿದರೆ ಯಾವ ಡೇಟಾ, ಯಾವ ಮಾದರಿಯಲ್ಲಾದರೂ ನೀವು ಪ್ರಾಮಾಣಿಕ ತೀರ್ಮಾನಕ್ಕೆ ಬರುತ್ತೀರಿ.